

基于序贯抽样与稳健决策的生产优化问题研究

摘要

在企业生产电子产品的过程中，一个成品次品往往由零配件次品、装配缺陷以及售后退换引起，并影响企业的经济成本与生产收益。因此，企业在有限成本下控制质量风险并提高利益具有重要的研究意义。本文针对企业的最优决策问题展开讨论。

针对问题一，本文建立了精确二项检验和 **e-process 序贯检验抽样模型**。通过 e-process 方案逐件抽检，每抽取一个零件后对应支持拒收的证据分数 E^+ 和支持接收的证据分数 E^- 进行更新。当 $E^+ \geq 20$ 时判定拒收（例如 2 件均为次品即可拒收），当 $E^- \geq 10$ 时判定接收（例如 34 件均无次品即可接收）。对模型进行检验与灵敏性分析后，得到模型真实次品率 $p = 5\%$ 时接收概率 99.64%， $p = 20\%$ 时拒收概率 99.95%，误拒与误收风险均控制在 5% 与 10% 以内。

针对问题二，本文首先建立起**检测与拆解决策模型**，接着建立**期望利润目标模型**，通过马尔可夫链建立**生产循环状态 Bellman 方程**，以 64 种策略全枚举方式求解生产循环中的最优决策。六种情形的结果分别为：零件 1 检测、零件 2 不检、成品不检、坏品拆解，利润 **18.84 元**；两零件均检、成品不检、坏品拆解，利润 **12.00 元**；零件 1 检测、零件 2 不检、成品检、坏品拆解，利润 **16.47 元**；两零件均检、成品检、坏品拆解，利润 **14.75 元**；零件 1 不检、零件 2 检测、成品不检、坏品拆解，利润 **14.96 元**；全部不检、坏品报废，利润 **21.68 元**。

针对问题三，本文在问题二的基础上建立起**多层级检测与拆解决策模型**和**分层质量状态摘要递推模型**，将装配过程抽象为 BOM 树形结构，并对全部 $2^{16} = 65536$ 种策略全枚举求解。得到最优策略为八种零配件与三个半成品均检测、各半成品次品均拆解、成品不检测、用户退回次品拆解，对应期望成本 **139.78 元**、期望利润 **60.22 元**。

针对问题四，本文对不确定次品率建立起**次品率置信上界估计模型**与**稳健决策重优化模型**，沿用问题一 e-process 反演得到各参数单侧置信上界，经 Bonferroni 校正构造系统置信集合，代入问题二、三模型在最坏情形下重优化。结果表明：95% 置信水平下问题二六情形利润由点估计的 18.84~21.68 元降至 5.23~14.91 元，其中情形 3、5、6 策略发生切换；问题三最优策略由成品不检测切换为检测，利润由 **60.22 元** 降至 **36.75 元**，体现了次品率不确定性对决策的显著影响。

最后，本文对所有模型进行了模型检验与灵敏度分析，确保得到的结果真实可靠。

关键词： 抽样检测 序贯决策 BOM 分层递推 稳健优化

一、问题重述

1.1 问题背景

在电子产品生产中，难免出现零配件不合格的情况。对企业生产来讲，次品往往意味着利益损失，因此，如何降低次品率与处理次品成为企业亟待解决的问题。

1.2 问题重述

在某企业中，成品由零配件 1 和零配件 2 组成。在成品中任一零配件不合格都会导致成品不合格。企业可以选择直接报废次品或者花费一定费用拆解次品。

本题要求在不同生产条件下建立数学模型，综合考虑产品质量、生产成本、销售收益和售后损失，为企业制定合理的检测、装配、拆解及调换决策，以获得较好的经济效益。

问题 1： 供应商声称某批零配件的次品率不超过给定的标称值。企业通过抽样检测判断是否接收这批零配件，费用由企业承担。要求设计一种检测次数尽可能少的抽样检测方案并明确不同置信水平下的接收或拒绝接收判定规则。

当标称次品率为 10% 时，研究满足 95% 与 90% 置信度的情况下的检测方案，并根据检测结果决定是否接收该批零配件。

问题 2： 已知两种零配件与成品次品率，企业需要确定完整的生产策略。决策内容包括：是否检测零配件、是否检测成品、是否拆解不合格成品，以及如何处理退回产品。

针对题目所给的六种情况分别制定产生方案，并计算对应的成本、收益和质量指标。

问题 3： 将问题二推广到多道工序和多个零配件的生产过程。针对两道工序、八个零配件的装配系统，确定各零配件、半成品和成品是否检测，以及不合格产品是否拆解，并给出相应的决策方案和评价指标。

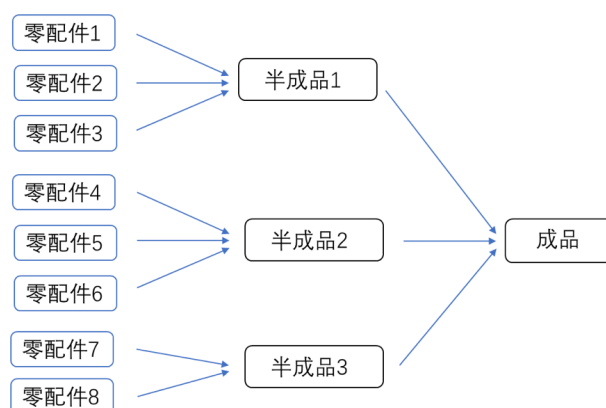


图 1 两道工序、八个零配件的组装结构

问题 4： 考虑到零配件、半成品和成品的次品率通常由抽样检测估计得到，具有一

定的不确定性。结合问题一的抽样方法，重新分析问题二和问题三，研究次品率估计误差对生产决策、成本、收益和产品质量的影响。

二、总体分析

2.1 问题一的分析

问题一要求设计一抽样检测方案用于判断是否接收供应商提供的零配件。本文综合采用概率模型与统计检验方法，将检测判定问题转化为可计算的数学模型^[6]。供应商声称次品率不超过标称值 $p = 10\%$ ，企业承担检测成本。因此，方案需同时满足两个目标：判定可靠且检测次数尽可能少。由于每次抽检结果只有合格与次品两种状态，单个零件的检测结果可抽象为 0-1 随机变量，因此 n 次独立抽检中出现的次品数服从二项分布。据此，本文首先建立起二项分布模型，通过判断当前样本 (n, k) 在次品率为 10% 假设下的概率，判断结果是否反常。对于至少 k 个次品的概率低于 5% 的情况，说明次品多的异常，选择拒收；对至多 k 个次品的概率低于 10%，说明次品少的异常，选择接收；否则说明证据不足，继续抽样。

假如直接使用二项分布做多次重复判断，每次重新计算会导致出现偶然显著的情况。同时，不能很好地完成题目对检测次数尽可能少的要求。为此，本文引入 e-progress 方法，建立起证据分数分别记录支持拒收与接收的证据强度^[2, 3]。每抽检一次后更新证据分数，当拒收证据分数满足题目要求时拒收，接受证据份数满足题目要求上接收，否则继续抽检下一样品。最终选择出合适的方案。

2.2 问题二的分析

问题二要求在生产过程中选择合适的检测与处理方案，使企业利益最大化。企业的生产过程具有循环性，若成品未检测而直接进入市场，可能因为不合格而被用户退回；若不合格成品选择拆解，拆出的零配件还可以重新进入检测与装配环节。因此，企业最终交付一件合格成品所需的成本，不仅包括第一次生产所需的费用，还包含退换货、拆解与重复生产产生的后续成本。而且由于次品丢弃、调换、检测的成本不一，六种情况最终的最优方案并不相同。

首先，本文选择将企业最终成功交付一件合格成品作为一个完整决策周期。由于成品可能被检测出不合格，也可能在售出后被用户退回，企业还需进行报废、拆解或重新生产，因此不能只比较一次生产的成本，而应计算整个周期内的期望总成本。接着，将零配件 1 和零配件 2 是否检测、成品是否检测以及不合格成品是否拆解分别设为决策变量。对每一种决策方案，先分析零配件进入装配环节的概率，再计算成品合格或不合格的概率，并将检测、装配、销售、调换和拆解等费用纳入成本计算。同时，保留已有的检测信息，确保已检测的零件不再复检，未检测的零件重新判断，借助 Bellman 完成从

当前状态到最终交付成品所需的期望成本。在此基础上，对所有可能的拆解或检测组合进行比较，并以期望成本最小、期望利润最大为最优方案。

2.3 问题三的分析

问题三在问题二的基础上把决策模型推广到多道工序和多个零配件的装配系统中。但问题三的生产结构较复杂，且最终决策由当前生产状态与后期决策相关。据此在问题二的基础上建立 BOM 结构对问题二进行推广，以全枚举结果选择最优策略。

2.4 问题四的分析

问题四在前两问的基础上需要在次品率未知的情况下求解。考虑到抽样误差的影响，需要将统计不确定性纳入生产决策考虑。直接使用点估计可能导致最优方案对参数扰动敏感，鲁棒优化理论表明可在目标性能与决策稳健性之间权衡^[4]。因此，本文首先用置信区间量化不确定性，由于次品率被低估会直接放大坏件进入装配、拆解与售后环节的风险，企业真正需要防范的是次品率偏高情形，而非偏低，故在给定风险水平下采用单侧置信上界，由于序贯抽样在证据充足时即停止抽检，因此本文沿用问题一的 e-progress 方法得到对应的置信上界。然而仅有单个参数的上界不足以保证整个生产系统的安全，因此通过 Bonferroni 校正构造系统置信集合，最后将该置信上界代入问题二与问题三的模型中，在置信集合上对候选策略进行最坏情形重优化确定决策。最后建立稳健决策检验模型判断当前信息是否支持稳定决策。

三、模型假设

为简化问题，本文做出以下假设：

1. 各零配件、半成品与成品的次品率在生产过程中保持稳定，同一批次内不随时间漂移。
2. 抽样检测完全准确，检测出的次品即为真实次品，不存在漏检与误判。
3. 不合格成品的拆解可完整回收其内部零配件，且回收件的质量状态在重新进入装配时与新品无差异。
4. 同一批次中各零配件与各工序的次品相互独立，半成品与成品的工序失效率仅对应合格输入条件下的装配缺陷。

四、符号说明

符号	含义	单位
p	实际次品率	概率
p_0	标称次品率	概率
n	抽样总数	个
k	样本中的次品数	个
E^+	支持次品率高于标称值的证据分数	—
S	成品销售价格	元
C	最终成功交付一件合格成品的期望总成本	元
π	期望利润	元
x_i	零配件 i 是否检测的决策变量	—
y	是否检测成品的决策变量	—
z	是否拆解不合格成品的决策变量	—
L	调换损失	元
p_j	半成品 j 的工序次品率	概率
y_j	半成品 j 是否检测的决策变量	—
z_j	半成品 j 不合格时是否拆解的决策变量	—
y_F	最终成品是否检测的决策变量	—
z_F	不合格成品是否拆解的决策变量	—
q	部件进入下一工序时的合格概率	概率
γ	稳健置信水平	—
U_j	第 j 个参数在置信水平 γ 下的单侧上界	概率

五、模型的建立和求解

5.1 问题一的模型建立与求解

问题一要求设计一种抽样检测方案，在满足抽样次数尽可能少且判定准确的情况下作出最优决策。因此简单的二项检验不能满足题目要求。基于此本文建立了以下两层递进的模型。

5.1.1 问题一的模型建立

1. 精确二项检验模型的建立

针对小样本条件下的验收问题，可直接基于二项分布构造精确统计检验^[2]。

首先设原假设 H_0 ：次品率 $p \leq 0.1$ 即供应商的说法成立；备择假设 H_1 ：次品率

$p > 0.1$ 即供应商的说法不成立。

每抽检一个零件，结果 X_i 服从 0-1 分布， $X_i = 1$ 表示次品， $X_i = 0$ 表示合格。抽检 n 个零件后，次品总数 $K = \sum_{i=1}^n X_i$ 服从二项分布 $B(n, p)$ 。

在供应商声称 $p_0 = 0.1$ 的假设下， $K \sim B(n, 0.1)$ ，即：

$$P(K = k) = \binom{n}{k} (0.1)^k (0.9)^{n-k} \quad (5.1)$$

接着进行拒收判断与接收判断。

拒收判断：计算当前样本 (n, k) 在 $p = 0.1$ 假设下出现至少 k 个次品的概率：

$$P_u = P(K \geq k \mid p = 0.1) = \sum_{i=k}^n \binom{n}{i} (0.1)^i (0.9)^{n-i} \quad (5.2)$$

若 $P_u < 5\%$ ，说明在次品率 = 10% 的假设下，当前结果过于反常，拒绝 H_0 ，判定为拒收。

接收判断：当认为次品率可能低于 10% 时，建立反向假设：原假设 H'_0 ：次品率 $p \geq 0.1$ ；备择假设 H'_1 ：次品率 $p < 0.1$ 。

计算至多 k 个次品的概率：

$$P_d = P(K \leq k \mid p = 0.1) = \sum_{i=0}^k \binom{n}{i} (0.1)^i (0.9)^{n-i} \quad (5.3)$$

若 $P_d < 10\%$ ，说明在“次品率 = 10%”的假设下，合格品多得异常，拒绝 H'_0 ，判定为接收。

对于任意 (n, k) ，存在三种可能：

- $P_u < 5\%$ ：拒收
- $P_d < 10\%$ ：接收
- 两者均不满足：证据不足，继续抽样

2. e-process 模型的建立

为减少不必要的检测次数，本文采用序贯决策思想，根据当前累计检验结果动态更新统计证据，并在满足预设判定条件时及时停止检测^[3]。

首先建立两个累计证据分数：

- E^+ ：支持次品率 $> 10\%$ 的证据强度，初始值为 1。
- E^- ：支持次品率 $< 10\%$ 的证据强度，初始值为 1。

每抽检一个零件，根据结果更新分数。而更新因子的确定基于以下考虑：

(1) 当抽到次品时，该结果对“次品率 $> 10\%$ ”提供强证据，因此 E^+ 乘以一个大于 1 的因子、 E^- 乘以一个小于 1 的因子。取因子 10 和 0.1，即观察到一次次品，相当于将支持“次品率 $> 10\%$ ”的证据强度提升为原来的 10 倍，同时将支持“次品率 $< 10\%$ ”的证据强度压缩为原来的 0.1 倍。

(2) 当抽到合格品时，该结果对“次品率 $< 10\%$ ”提供弱证据，因此 E^- 乘以一个大于 1 的因子、 E^+ 乘以一个小于 1 的因子。取因子 $10/9$ 和 0.9 ，即观察到一次合格品，相当于将支持“次品率 $< 10\%$ ”的证据强度提升为原来的 $10/9$ 倍，同时将支持“次品率 $> 10\%$ ”的证据强度压缩为原来的 0.9 倍。

因子取值由 Ville 不等式确定：原假设 $p \leq 0.1$ 成立时， $P(E^+ \geq c) \leq 1/c$ 。取 $c = 20$ ，则误拒风险 $P(E^+ \geq 20) \leq 5\%$ ，对应 95% 置信度；取 $c = 10$ ，则误收风险 $P(E^- \geq 10) \leq 10\%$ ，对应 90% 置信度。

据此得到更新规则与停止规则。

更新规则：

当抽到次品时：

$$E^+ \leftarrow E^+ \times 10, \quad E^- \leftarrow E^- \times 0.1 \quad (5.4)$$

当抽到合格品时：

$$E^- \leftarrow E^- \times \frac{10}{9}, \quad E^+ \leftarrow E^+ \times 0.9 \quad (5.5)$$

停止规则：

- 若 $E^+ \geq 20$ ，则拒收
- 若 $E^- \geq 10$ ，则接收
- 否则，继续抽取下一个零件。

通过 e-progress 的辅助，可以求得第一问的企业最佳抽样检测方案。

5.1.2 问题一的求解结果

根据所建立的 e-process 模型，得到序贯抽样边界如表 2 所示，表中给出了不同抽样量 n 下判定接收或拒收的临界次品数 k 。

表 2 序贯抽样边界（部分）

n	接收边界 k	拒收边界 k
2	—	2
34	0	10
40	0	11
50	0	13
80	2	18
100	3	21
160	7	30
200	10	36

表 2（续表）

n	接收边界 k	拒收边界 k
500	33	75

从表中可以看出，序贯抽样边界随着 n 的增大而逐渐放宽，最早在 $n = 2$ 时即可达到拒收边界，最早在 $n = 34$ 时达到接收边界。

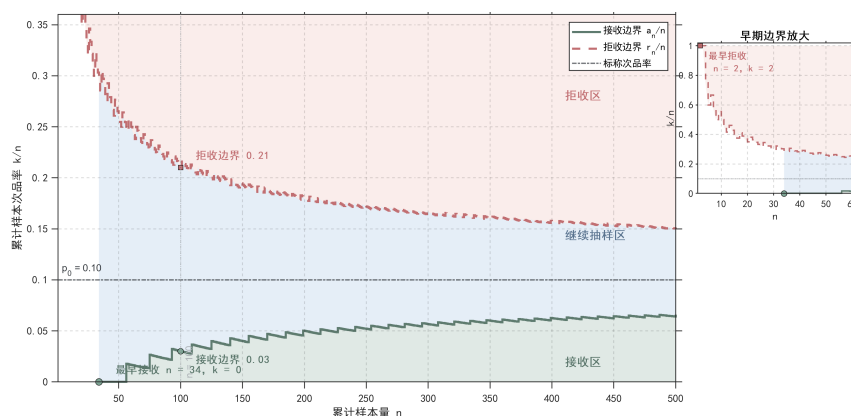


图 2 序贯停止边界与三决策区域

序贯决策域含接收、继续抽样、拒收三区。小样本阶段边界较宽以防误判；随 n 增大，接收边界上移、拒收边界下移，共同逼近标称次品率 $p_0 = 0.10$ 。最早拒收在 $n = 2, k = 2$ ，最早接收在 $n = 34, k = 0$ 。

此外，将序贯停止边界与固定样本精确二项检验边界进行对比，结果如图 3 所示：

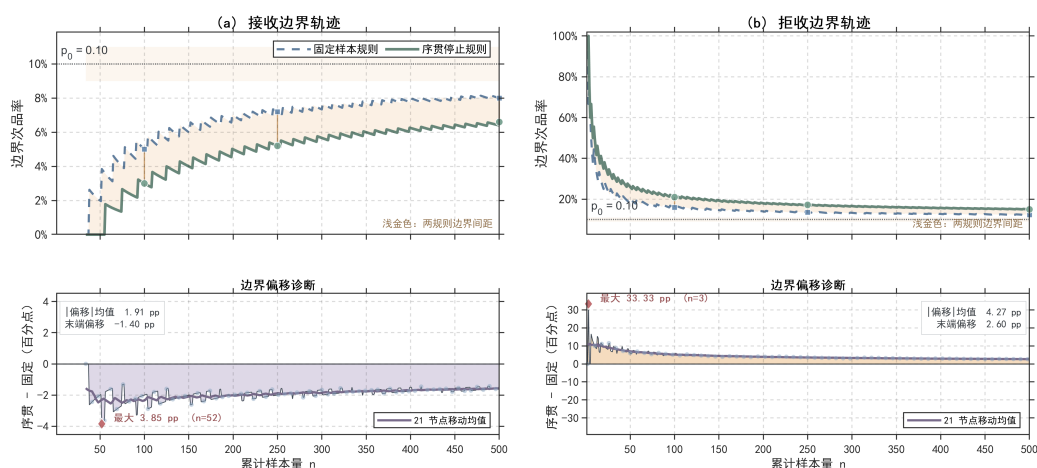


图 3 固定样本与序贯停止边界对比

固定样本规则与序贯规则的边界变化趋势一致，但在小样本阶段存在一定差异；随着样本量增加，两者的边界逐渐接近。

同时，采用动态概率递推方法对方案进行性能评估。递推流程如图 4所示。

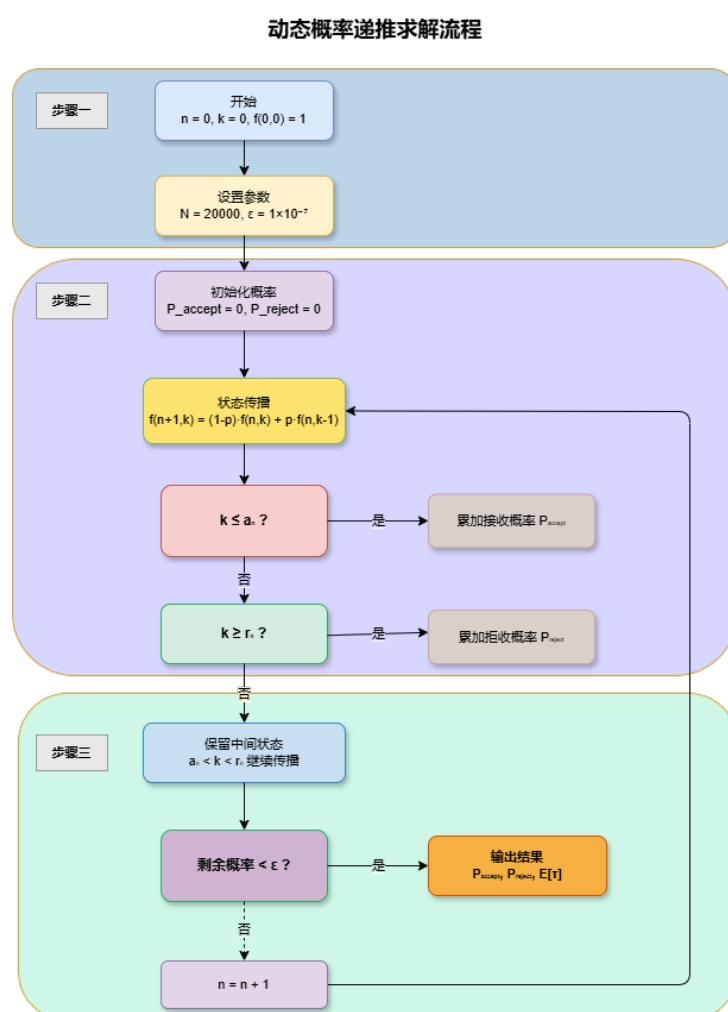


图 4 动态概率递推求解流程

设定真实次品率 p 分别取 5%、8%、10%、12%、20%。递推结果如表 3所示：

表 3 不同真实次品率下的方案性能

真实次品率 p	接收概率	拒收概率	条件期望样本量
5%	99.64%	0.36%	161
8%	98.76%	1.24%	1292
10%	9.29%	2.99%	677
12%	2.16%	97.84%	2846
20%	0.05%	99.95%	103

结果分析：当 $p = 5\%$ 和 $p = 20\%$ 时，真实次品率远离标称值 $p_0 = 0.10$ ，证据分数快速向拒收或接收方向积累，方案能以较少样本做出正确判断；当 $p = 10\%$ 时，真实

次品率恰落在判定边界，此时两类假设的似然比接近 1，证据分数在阈值附近长期徘徊，87.72% 的路径在 20000 步内未做出判定。这说明越接近质量判定边界，两类假设越难区分，需要更多样本积累证据，符合统计检验在临界点功效降低的一般规律。

真实次品率对检验性能与检测成本的影响如图 5 所示：

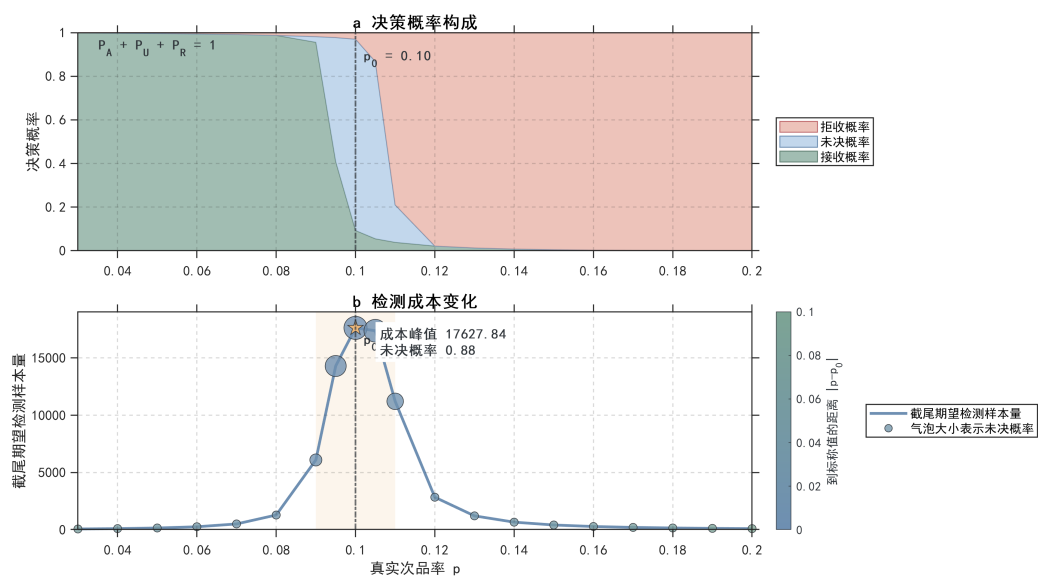


图 5 真实次品率对检验性能与检测成本的影响

检测样本量关于真实次品率呈阈值峰值结构：标称值 $p_0 = 0.10$ 附近两类假设似然比接近 1，证据分数在阈值附近长期徘徊，单条路径难以及时终止，截尾期望样本量达约 17627；远离阈值后，真实次品率与标称值差异增大，单次抽样的证据增量显著，证据分数快速越过判定线，期望样本量迅速下降。相比固定样本方案不论批次优劣一律抽检相同数量，序贯机制在保持临界区域谨慎性的同时，对易判别批次提前停止，显著降低了平均检测次数与抽样成本。

5.1.3 问题一的模型检验与灵敏度分析

为进一步验证方案的可靠性，从以下两个维度进行检验：

(1) 错误率控制检验

为检验两类错误风险是否被有效控制，绘制误拒与误收风险随真实次品率的变化曲线，如图 6 所示。

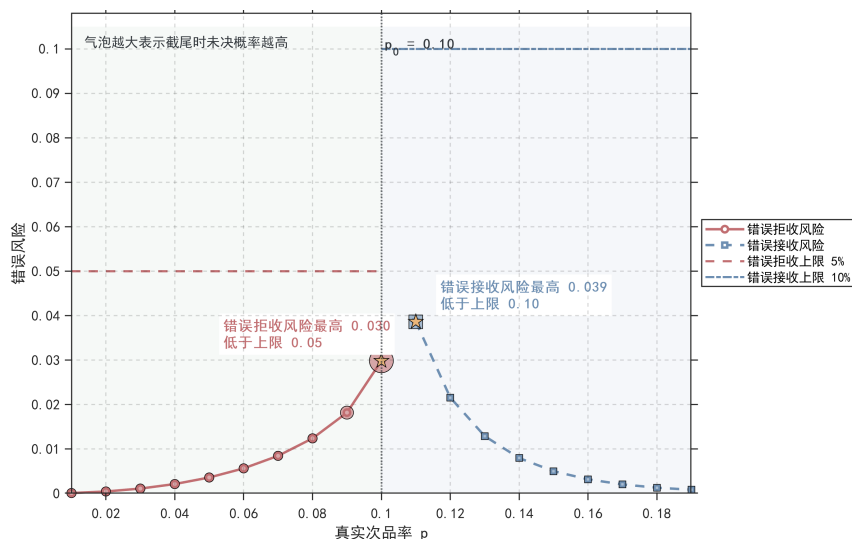


图 6 序贯检验错误风险控制验证

由图可见，在 $p \leq 10\%$ 范围内误拒风险均未超过 5% 上限（ $p = 5\%$ 时 0.36%， $p = 10\%$ 时 2.99%）；在 $p \geq 10\%$ 范围内误收风险均未超过 10% 上限（ $p = 12\%$ 时 2.16%， $p = 20\%$ 时 0.05%）。全部网格点通过理论上限检查，验证了方案对两类错误的有效控制。

(2) 样本效率检验

方案呈现出明显的边界效应， $p = 5\%$ 时条件期望样本量仅 161， $p = 20\%$ 时仅 103，而 $p = 10\%$ 附近则有 87.72% 的路径在 20000 步内无法判定，表明方案在证据充分时尽早停止、边界模糊时充分采样之间取得了合理平衡。

检验结果表明，该方案满足题目要求的检测次数尽可能少和 95% 置信度拒收、90% 置信度接收的双重目标，可用于实际生产中的零配件验收环节。

(3) 灵敏度分析

进一步分析真实次品率与置信水平对检测成本的联合影响，如图 7 所示：

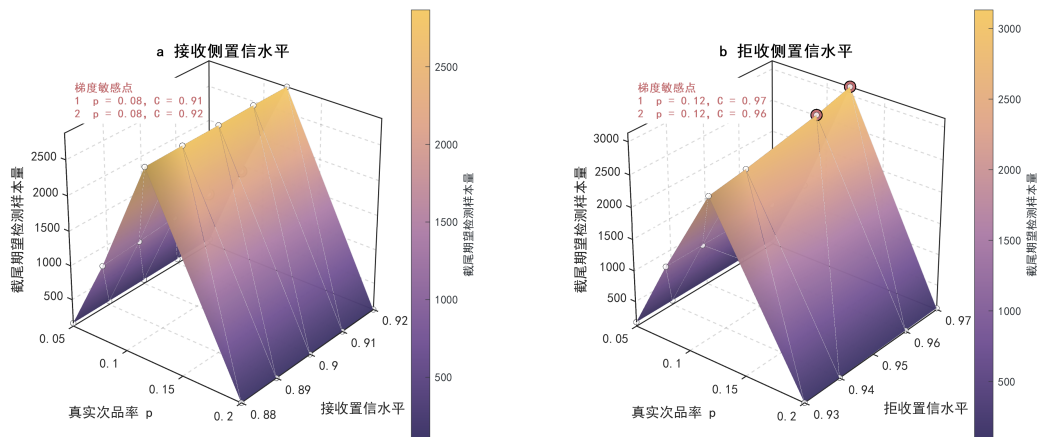


图 7 真实次品率与置信水平对检测成本的联合影响

联合敏感性分析表明，检测成本不仅受真实次品率影响，也会随两侧置信水平发生局部陡变。基于置信水平方向局部梯度识别出的敏感节点主要位于阈值邻域两侧，说明提高置信要求会在难判别区域显著增加检测成本。该结果为风险水平与抽检代价之间的参数权衡提供了依据。

5.1.4 问题一的结论

针对问题一，本文采用 e-process 序贯抽样方案。得到如下**结果**：

(1) 在 95% 信度下拒收：当 $E^+ \geq 20$ 时判定拒收。例如抽检 2 个均为次品即可拒收；抽检 100 个时次品数达到 21 个则拒收。

(2) 在 90% 信度下接收：当 $E^- \geq 10$ 时判定接收。例如抽检 34 个均无次品即可接收；抽检 100 个时次品数不超过 3 个则接收。

模型检验显示，真实次品率 $p = 5\%$ 时，接收概率 99.64%，条件期望样本量 161； $p = 20\%$ 时，拒收概率 99.95%，条件期望样本量 103。 $p = 10\%$ 时误拒风险 2.99%， $p = 12\%$ 时误收风险 2.16%，两类错误均控制在题目要求的 5% 和 10% 以内。

5.2 问题二的模型建立与求解

问题二中需要求解在生产过程中涉及循环的零配件检测与处理方案，使得企业最终交付一件合格产品时的平均利润最大。针对这一过程，本文建立生产循环检测与拆解决策优化模型。企业需要对零配件是否检测、对成品是否检测，以及对不合格成品是否进行拆解。不同决策会影响检测费用、装配费用、报废损失、调换损失和零配件的重复利用价值，并同时受到多种因素的影响。基于此，本文建立起以下模型求解。

5.2.1 问题二的模型建立

1. 检测与拆解决策模型的建立

本问题的决策并非一次性完成，而是随着零配件检测结果、装配结果和成品处置结果逐步展开。为此，本文构造六个二元决策变量，统一表示为设零配件检测为 x_1, x_2 、成品检测为 y 、不合格成品拆解为 z 、回收件复检为 r_1, r_2 ，取值均为 1（执行）或 0（不执行）。一个完整策略表示为

$$\mathbf{x} = (x_1, x_2, y, z, r_1, r_2) \in \{0, 1\}^6 \quad (5.6)$$

其中前四个变量决定首次生产的检测与不合格品处置，后两个变量决定拆解回收后的零配件处理。

2. 期望利润目标模型的建立

本文选择将企业成功交付一件合格成品作为完整决策周期。但由于成品可能在检测环节判断为不合格，也可能因为售后问题被退回，因此不能仅比较单次生产，必须将整

个决策周期内的期望总成本作为评价标准。检测本身会产生额外成本，最优检测方案不能仅以降低次品率为目标，还需综合权衡检测成本与质量损失之间的经济关系^[7]。

故设市场售价为 S ，企业最终成功交付一件合格成品所需的期望总成本为 C ，则期望利润为：

$$\max_{\mathbf{x} \in \{0,1\}^6} \pi(\mathbf{x}) = S - C(\mathbf{x}) \quad (5.7)$$

其中 $\mathbf{x} = (x_1, x_2, y, z, r_1, r_2)$ 为策略向量， $C(\mathbf{x})$ 为该策略下的期望总成本，包括零配件购买成本、检测成本、装配成本、成品检测成本、调换损失、拆解成本和报废成本。对于每种策略，选取 π 最大的作为最佳策略。

对于每一个有效策略，模型按照以下阶段依次计算其期望总成本，如图 8 所示：

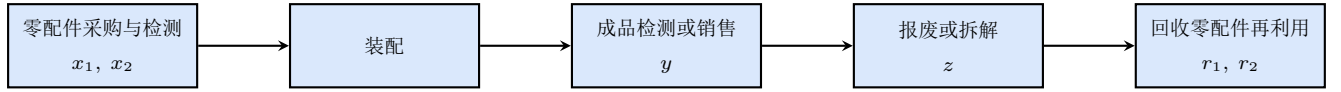


图 8 生产周期中各阶段的成本计算流程

理论上，六个二元变量构成 $2^6 = 64$ 个原始策略。需要注意的是，当 $z = 0$ 时，不合格成品直接报废，不会进入拆解回收环节，此时 r_1 和 r_2 不会被实际触发，属于冗余变量。因此，模型首先对 64 个原始策略进行统一编码，并将 $z = 0$ 时的回收决策规范化为

$$r_1 = r_2 = 0 \quad (5.8)$$

最终得到 40 个具有实际区别的有效策略。

3. 成品质量概率模型的建立

题目给出的成品次品率 p_3 是指两个零配件均合格时，装配过程本身产生次品的概率。因此成品合格率需根据零配件检测情况分别计算：

$$P_{\text{合格}} = \begin{cases} (1 - p_1)(1 - p_2)(1 - p_3), & \text{两个零配件均未检测} \\ (1 - p_2)(1 - p_3), & \text{零配件 1 已检测合格} \\ 1 - p_3, & \text{两个零配件均已检测合格} \end{cases} \quad (5.9)$$

4. 生产循环状态 Bellman 转移模型的建立

由于生产具有循环性，后期决策是当前状态与后续期望成本综合决定，据此将整个生产过程建模为马尔可夫链，每个状态表示企业当前拥有的零件质量信息。

对于每个状态，定义 V_s 为从该状态到最终成功交付的期望总成本，建立 Bellman 方程：

$$V_s = g(s, a) + \sum_{s'} P(s'|s, a) V_{s'} \quad (5.10)$$

其中， $g(s, a)$ 表示系统处于当前状态 s 并采取动作 a 时产生的即时成本； s' 表示一次决策后可能到达的下一状态， $P(s'|s, a)$ 表示在当前状态 s 下采取动作 a 后转移至下一状态 s' 的概率； $V_{s'}$ 表示从下一状态 s' 继续运行直至最终成功交付所需的期望成本。

5. 多方案期望利润优化模型的建立

由于决策变量均为 0-1 变量，六位策略空间共 64 种候选方案。采用全枚举方法逐一求解每种策略的期望成本，选取期望利润最高的方案作为最优策略。

5.2.2 问题二的求解结果

对每一情形枚举全部候选策略并比较期望利润，得到的最优策略及结果如表 4 所示。

表 4 六种情形的最优决策结果

情形	零件 1 检测	零件 2 检测	成品检 测	拆解	期望成 本	期望利 润	次优策 略	利润 差
1	是	否	否	是	37.16	18.84	0001	0.31
2	是	是	否	是	44.00	12.00	1001	1.48
3	是	否	是	是	39.53	16.47	0011	0.05
4	是	是	是	是	41.25	14.75	0111	1.96
5	否	是	否	是	41.04	14.96	0111	0.31
6	否	否	否	否	34.32	21.68	1000	0.35

对六种情形进行分析，得到图 9。

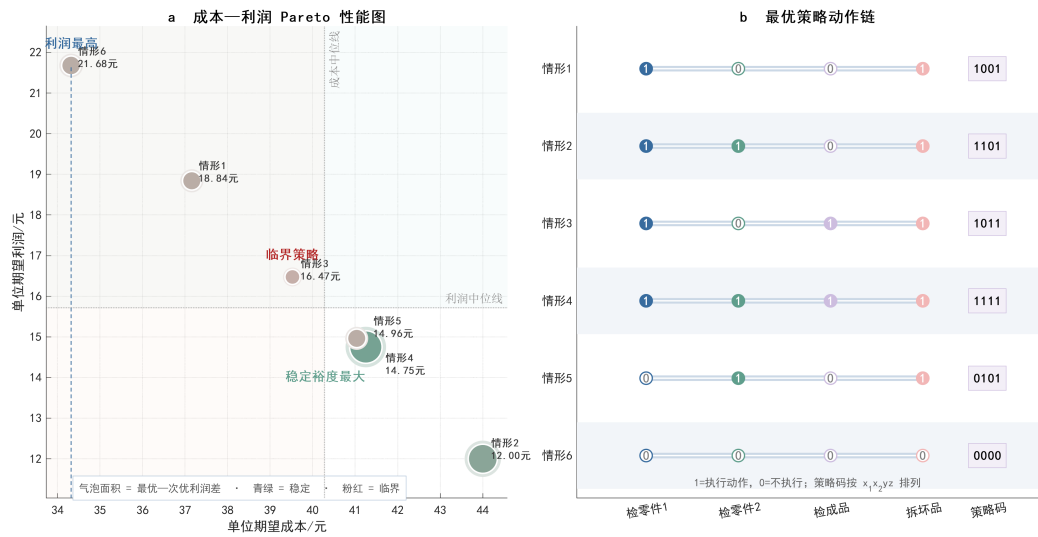


图 9 六种生产情形最优决策的成本—利润表现与策略结构

由图 9 可见，六种情形最优利润为 12.00 ~ 21.68 元。情形 6 零件次品率仅 5%，次品流出概率低，全不检、坏品报废即可截断损失，成本最低、利润最高，表明低次品率下过度检测反而增加支出；情形 2 两零件次品率均为 20%，次品流入装配的概率高，必须开启检测以避免更大损失，检测投入增加使利润最低。情形 4 最优与次优利润差最大，

策略优势显著、稳定性强；情形3与次优利润接近，最优策略易受参数变化影响，决策灵敏度较高。

为深入分析各策略的经济结构，图10给出六种情形下的单位期望成本及利润空间。

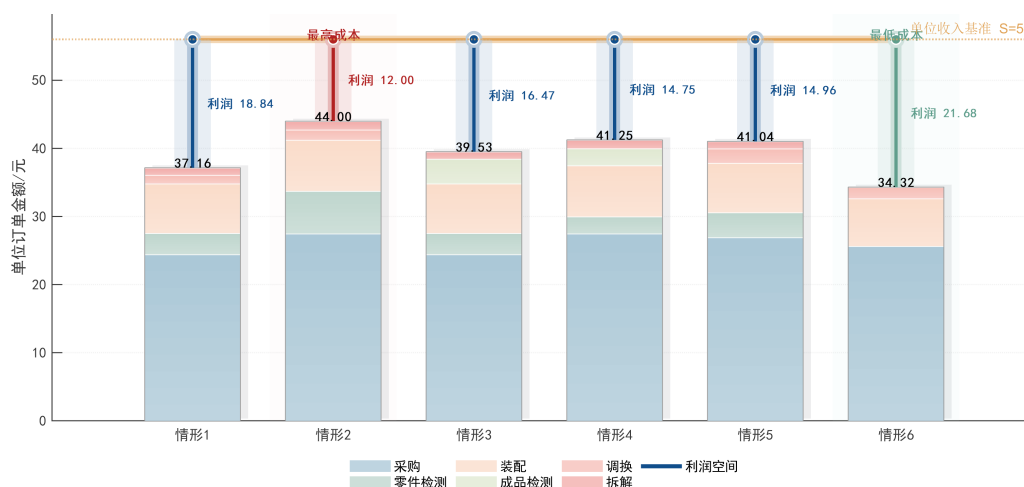


图10 六种情形最优策略的单位期望成本构成及利润空间

由图可见，各情形期望成本均以采购成本为主体，装配与检测费用次之，调换与拆解成本占比最低；成品检测虽产生额外费用，但能拦截不合格品流入市场、显著减少调换损失，在次品率较高情形下净收益为正。情形6因次品率仅5%，采购浪费少、调换风险低，总成本最低、利润空间最大，表明低次品率下过度检测与拆解的边际收益不足以抵消其费用，并非越严越好。

5.2.3 问题二的模型检验与结果分析

为验证模型求解的正确性与可靠性，本文从解析退化、蒙特卡洛模拟及极端情形三个维度进行交叉检验。

(1) 解析退化验证

将模型退化为无拆解、无循环情形，并与封闭解析式比较。六种情形共96个策略均通过验证，最大相对误差仅为 5.119×10^{-16} ，表明模型计算正确。

(2) 蒙特卡洛模拟验证

对每种情形生成 10^5 笔随机订单，模拟生产全过程的期望利润，结果如图11所示。

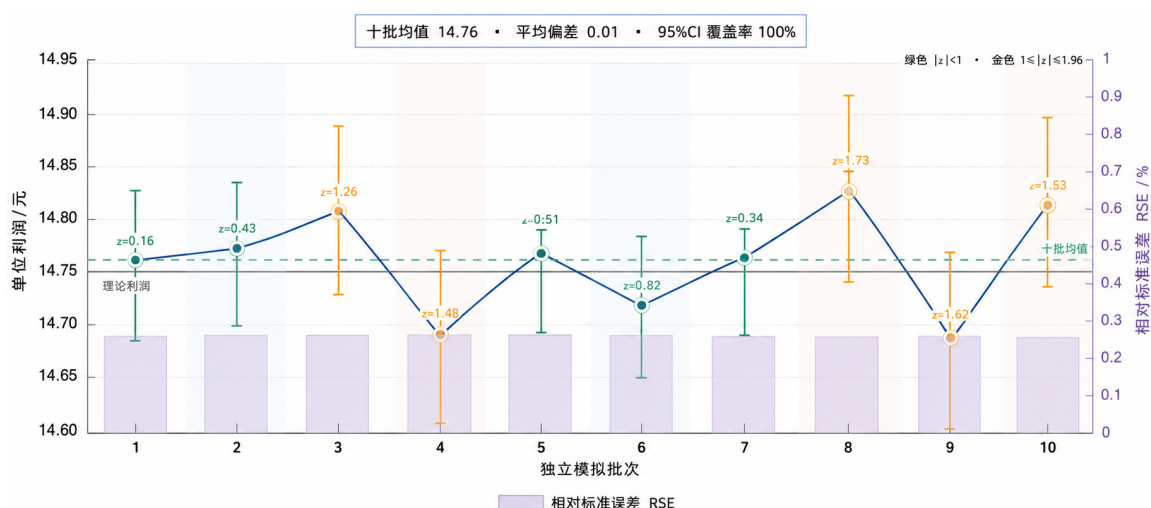


图 11 情形 4 理论利润的 Monte Carlo 多批次验证结果

除情形 4 外，各情形理论利润均落入模拟 95% 置信区间内。情形 4 因高次品率与高调换损失并存，次品流出属小概率稀有事件，蒙特卡洛对稀有路径采样精度有限，模拟均值 14.87 元与理论值 14.75 元偏差 0.12 元，处于随机波动范围内，不改变策略结论。

(3) 极端情形检验

构造若干极端参数场景检验模型行为，结果如表 5 所示。

表 5 极端情形检验结果

检验场景	理论预期	模型输出
零缺陷且检测费 > 0	全不检	0000（全不检）
调换损失极高	检测成品	1011（检成品）
拆解费用极高	不拆解、报废	1000（报废）
高风险且检测免费	全检	1111（全检）
未知坏回收件永不检测	永久循环、不可行	不可行（ $\rho = 1$ ）

此外，当两零配件均已证合格时，单轮装配的成品合格率应恰为 $1 - p_3 = 0.90$ ，模型输出与实际一致；而未知坏回收件永不检测的策略因其暂态矩阵谱半径 $\rho \geq 1$ 、存在期望成本发散的永久循环，被判定为不可行。

综合三项检验可见，模型在解析退化、蒙特卡洛模拟、极端情形检验中均表现良好，求解结果可靠，可为企业生产决策提供依据。

(4) 灵敏度分析

以基准情形 1 为例，对关键参数进行一维扫描，考察最优策略随参数变化的临界切换点，结果如表 6 所示。

表 6 情形 1 关键参数的灵敏度临界值

参数	临界值	临界前策略	临界后策略	当前参数下的结论
零件 1 次品率 p_1	8.37%	0001	1001	$p_1 = 10\% > 8.37\%$, 检测件 1
零件 2 次品率 p_2	13.71%	1001	1101	$p_2 = 10\% < 13.71\%$, 不检测件 2
零件 1 检测成本 b_1	2.43	1001	0001	$b_1 = 2 < 2.43$, 检测件 1
零件 2 检测成本 b_2	2.10	1101	1001	$b_2 = 3 > 2.10$, 不检测件 2
调换损失 L	13.30	1001	1101	$L = 6 < 13.30$, 不检测成品
拆解成本 d	12.30	1001	1101	$d = 5 < 12.30$, 选择拆解

当前参数下, 零件 1 次品率 $p_1 = 10\%$ 高于临界值 8.37%, 表明其次品一旦流入装配将引发拆解与调换的连锁损失, 该后续损失已超过单件检测成本, 故检测零件 1 有利可图; 零件 2 次品率 $p_2 = 10\%$ 低于临界值 13.71%, 其次品风险尚处于可容忍区间, 检测投入无法被避免的损失抵消, 故不值得检测; 调换损失 $L = 6$ 元低于临界值 13.3 元, 说明售后退换的期望代价有限, 成品检测所减少的调换损失不足以覆盖其检测费用, 故无需检测成品。

三个临界值将参数空间划分为检测获益与检测亏损两个区域, 企业当前参数恰好落在零件 1 应检、零件 2 与成品不应检的区域。各临界结论与最优策略完全一致, 验证了解的合理性。

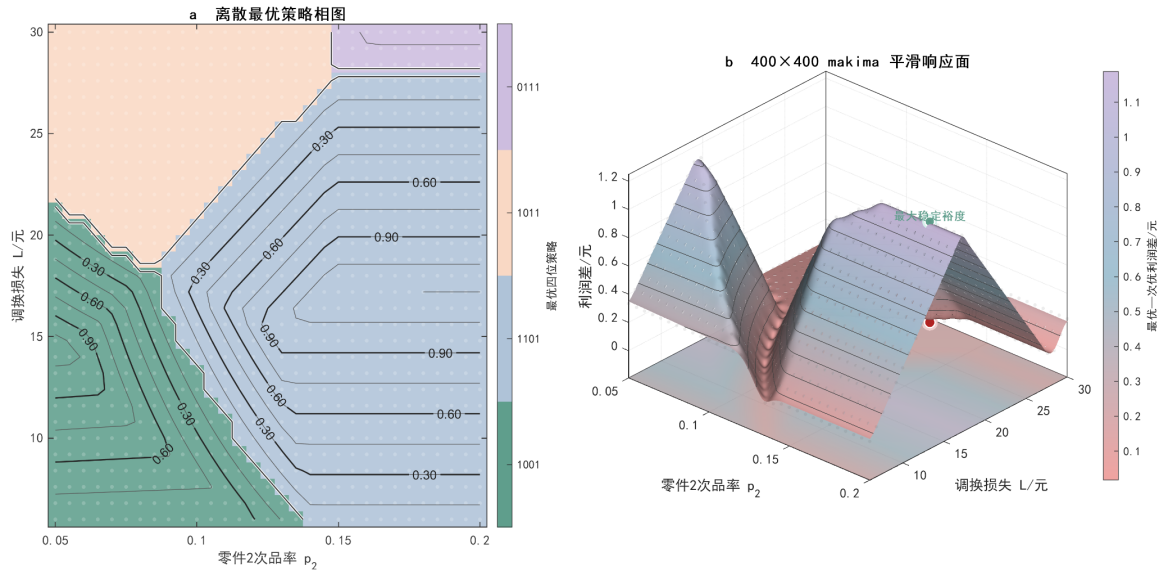


图 12 情形 1 中零件 2 次品率与调换损失共同作用下的策略分区及稳定裕度

图 12 表明, 零件 2 次品率与调换损失 L 的变化会共同推动最优策略切换; 在策略分界线附近, 利润差较小, 决策对参数误差更为敏感。



图 13 六种情形最优与次优策略的单位期望利润比较

图 13 进一步比较六种情形最优策略与次优策略的单位期望利润。六种情形的利润差依次为 0.31、1.48、0.04、1.96、0.31、0.35 元。其中情形 4 的利润差最大，策略 1111 相对次优策略具有较清晰的优势；情形 3 的利润差最小，最优策略 1011 与次优策略 0011 在经济表现上几乎相当，属于临界情形，对情形 3 给出确定性策略结论时应附带临界性说明。

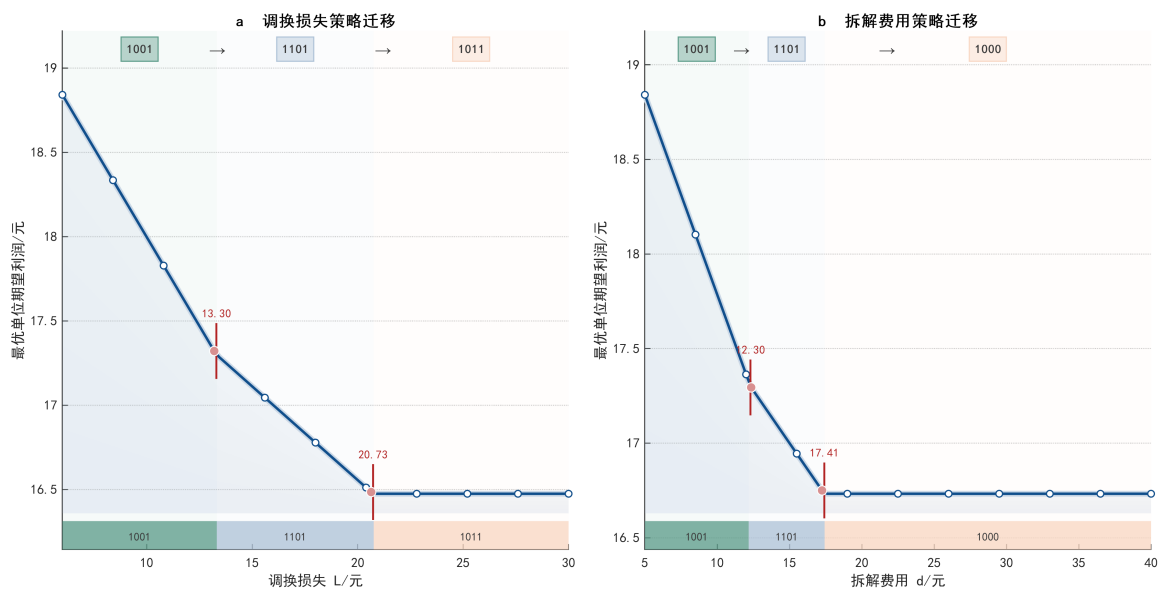


图 14 情形 1 中调换损失与拆解费用变化下的最优策略切换

图 14 展示单参数变化下的策略迁移。当调换损失 L 增加时，最优策略依次发生 $1001 \rightarrow 1101 \rightarrow 1011$ 的迁移，对应阈值约 13.30 元和 20.73 元；当拆解费用增加时，策略在 12.30 元附近由 1001 切换为 1101，并在 17.41 元附近进一步切换为 1000，反映出拆

解回收价值下降后继续拆解不再划算。在两处临界阈值附近，两侧策略利润近似相等，实际应用时应在阈值附近设置缓冲区，避免参数轻微波动造成频繁切换。

5.2.4 问题二的结论

针对问题二，本文得到的六种情形最优方案如表 7所示。

表 7 问题二六种情形最优决策与经济指标汇总

情形	决策	期望利润/元
6	全不检、坏品报废	21.68
1	检零件 1、不检成品、拆解	18.84
3	检零件 1、检成品、拆解	16.47
5	检零件 2、不检成品、拆解	14.96
4	两零件与成品均检、拆解	14.75
2	两零件均检、不检成品、拆解	12.00

通过模型检验与灵敏度分析，验证了本文模型具有较好的稳定性：Bellman 矩阵解与解析闭式解在全部 48 个策略格点上最大误差 4.3×10^{-14} ，灵敏度分析显示各情形最优策略在参数邻域内保持不变。

5.3 问题三的模型建立与求解

5.3.1 问题三的模型建立

问题三的生产系统包含八种零配件、三个半成品和一个成品, 其装配关系如图 1所示。与问题二的单层闭环不同, 本题的决策需沿 BOM 树形结构逐层展开。

1. 多层级检测与拆解决策模型的建立

题目共涉及 16 个 0-1 决策变量, 分别对应零配件检测 x_i ($i = 1, \dots, 8$)、半成品检测 y_j 与拆解 z_j ($j = 1, 2, 3$)、成品检测 y_F 与拆解 z_F , 取值均为 1（执行）或 0（不执行）。完整策略编码为

$$\boldsymbol{x} = (x_1, \dots, x_8, y_1, y_2, y_3, z_1, z_2, z_3, y_F, z_F) \in \{0, 1\}^{16} \tag{5.11}$$

总策略空间为 $2^{16} = 65536$ 种方案。

2. 分层质量状态摘要递推模型的建立

本问题的完整决策周期仍然以企业最终成功交付一件合格成品为口径, 期望利润为市场售价减去期望总成本。

问题三具有明显的 BOM 层级结构^[5], 若展开所有节点状态, 模型复杂度会迅速增加。因此, 本文采用分层状态压缩递推方法, 将各子树的成本、质量等关键信息汇总后传递至上层。各阶段质量状态与决策结果形成树形事件演化结构, 沿叶节点向根节点递推概率与期望收益^[8]。其网络拓扑与状态递推框架如图 15 所示。

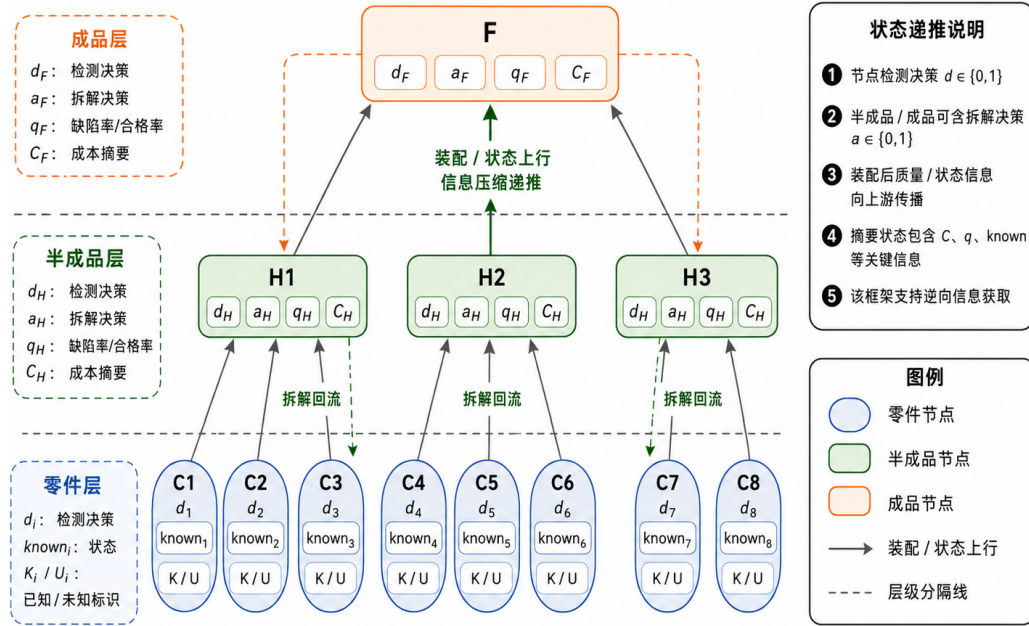


图 15 BOM 网络拓扑与状态递推框架

图中八类零配件 C_1 — C_8 分别进入三个半成品 H_1 — H_3 , 随后共同装配为成品 F 。每个节点同时记录质量状态与成本状态, 定义节点状态描述为

$$\mathcal{D} = (C, q, \ell, K, D, b) \quad (5.12)$$

其中各分量含义如下:

- C : 按当前策略获得一个该节点输出的期望成本;
- q : 该输出为合格品的概率;
- ℓ : 是否已确认该节点必然合格;
- K : 从头获得一个已知合格节点的期望成本;
- D : 节点被确认不合格后修复为已知合格的期望成本;
- b : 该节点单次检测费用。

节点状态的流转规则为: 完成检测后质量由未知转为已知, 发现不合格品时可选择报废或拆解; 拆解所得未知回收件依据上级产品已失败的信息, 通过贝叶斯条件概率更新其良品概率后返回相应装配环节。

零配件处理：对于基础零件 i ，令 x_i 表示是否检测，则其期望成本、合格概率和质量信息状态分别为

$$\begin{aligned} C_i(x_i) &= \begin{cases} a_i, & x_i = 0, \\ \frac{a_i + b_i}{1 - p_i}, & x_i = 1, \end{cases} \\ q_i(x_i) &= \begin{cases} 1 - p_i, & x_i = 0, \\ 1, & x_i = 1, \end{cases} \\ \ell_i(x_i) &= \begin{cases} 0, & x_i = 0, \\ 1, & x_i = 1. \end{cases} \end{aligned} \quad (5.13)$$

其中， C_i 为获得零件 i 的期望成本， q_i 为其合格概率， $\ell_i = 1$ 表示该零件已被确认合格， $\ell_i = 0$ 表示其质量仍未知。

半成品层处理：设半成品 j 的输入集合为 $\mathcal{I}_j$ ，其状态描述为 $\{\mathcal{D}_k : k \in \mathcal{I}_j\}$ 。令 p_j 、 a_j 、 b_j 和 d_j 分别表示工序次品率、装配费、检测费和拆解费，则

$$g_j = (1 - p_j) \prod_{k \in \mathcal{I}_j} q_k \quad (5.14)$$

$$B_j = a_j + \sum_{k \in \mathcal{I}_j} C_k \quad (5.15)$$

当 $z_j = 0$ 时，半成品报废并重新生产，直至获得已知合格输出：

$$K_j = \frac{B_j + b_j}{g_j}. \quad (5.16)$$

令 H_j 表示半成品 j 合格， $\bar{H}_j$ 表示半成品 j 不合格；令 $Z_u = 1$ 表示输入 u 合格， $Z_u = 0$ 表示输入 u 不合格。若 $z_j = 1$ ，则拆解并更新未知输入的质量概率：

$$r_{u|j} = P(Z_u = 0 \mid \bar{H}_j) \quad (5.17)$$

$$r_{u|j} = \frac{1 - q_u}{1 - g_j}. \quad (5.18)$$

记回收输入的认证成本为 $R_u(r_{u|j})$ ，工序自身失败的循环修复成本为 T_j ，则

$$D_j = d_j + \sum_{u \in \mathcal{I}_j} R_u(r_{u|j}) + T_j. \quad (5.19)$$

成品层处理：设最终成品 F 的输入集合为 $\mathcal{I}_F$ ，其状态描述为 $\{\mathcal{D}_k : k \in \mathcal{I}_F\}$ 。令 p_F 、 a_F 、 b_F 和 L 分别表示成品工序次品率、装配费、检测费和调换损失，则

$$g_F = (1 - p_F) \prod_{k \in \mathcal{I}_F} q_k, \quad B_F = a_F + \sum_{k \in \mathcal{I}_F} C_k. \quad (5.20)$$

令 K_F 和 D_F 分别表示成品报废和拆解后的恢复成本，则失败后的处理成本为

$$W_F(z_F) = (1 - z_F)K_F + z_FD_F. \quad (5.21)$$

将成品检测决策 y_F 、售后调换和失败后的循环统一计入，可得

$$C_F = \frac{B_F + y_F b_F + (1 - g_F)[(1 - y_F)L + W_F(z_F)]}{g_F}. \quad (5.22)$$

相应的期望利润为

$$\pi_F = S - C_F. \quad (5.23)$$

3. 全局最优方案的期望利润优化模型的建立

由于候选策略仅有 65536 种，本文采用全枚举结合 BOM 递推模型计算各方案的期望利润，并选取利润最大的方案作为全局最优解。

5.3.2 问题三的求解结果

对问题三的全部 $2^{16} = 65536$ 种策略进行全枚举求解，选取期望利润最大的方案作为最优策略。得到的最优策略为 **1111111111111101**，对应的单位期望成本为 139.78 元，单位期望利润为 60.22 元。最优策略的各层级决策如表 8 所示。

表 8 问题三的最优策略及经济含义

决策层	最优取值	经济含义
零配件检测	11111111	八种零配件全部检测，阻断质量风险向下游传播
半成品检测	111	三个半成品全部检测，及时截获工序次品
半成品拆解	111	不合格半成品均拆解，回收合格零配件
成品检测	0	不做出厂检测，检测边际成本高于可避免损失
成品拆解	1	退回成品拆解，回收合格半成品

如表 8 所示，最优策略在前端零配件与半成品层全面检测、各次品均拆解回收，唯独成品层不检出厂检测。原因在于半成品均已合格后，成品层仅剩 10% 装配缺陷风险，检测可避免的期望调换损失 $0.1 \times 40 = 4$ 元小于 6 元检测费，故成品不检、退回后拆解。这表明检测应布置在能阻断后续损失的关键层级，而非盲目全检。

将最优策略的 16 个二元决策映射回 BOM 节点，得到可执行的最优检测—拆解决策网络，如图 16 所示。

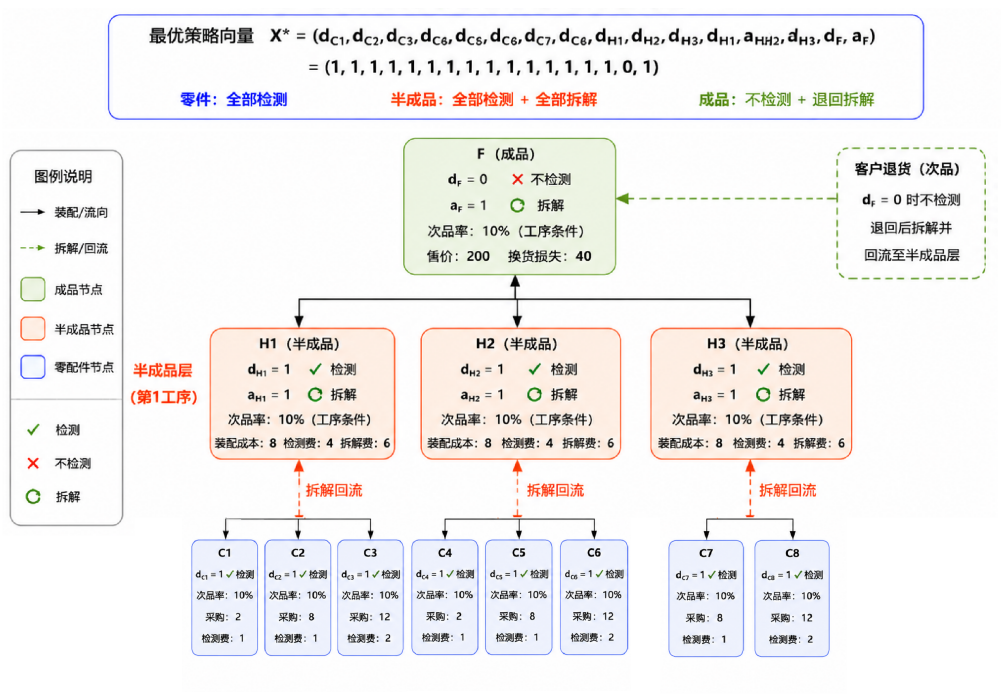


图 16 最优检测—拆解策略决策图

在最优策略下，期望总成本为 139.78 元，其成本结构分解如图 17所示。

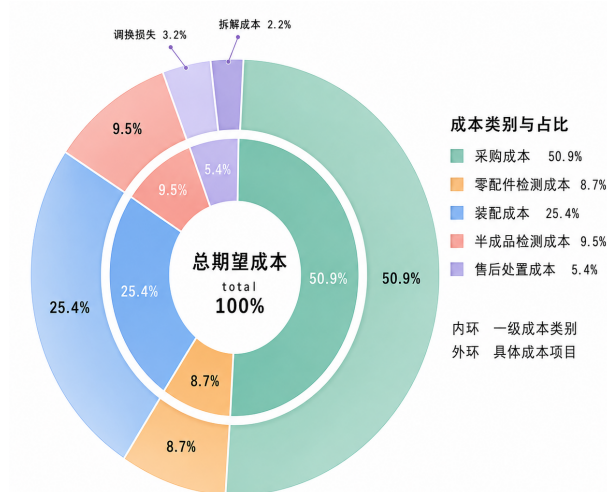


图 17 最优策略下的检测成本结构分解

其中采购与装配成本合计占 76.31%，构成成本主体；检测费用约 18.28%，以较小前置投入减少下游浪费与调换损失；调换与拆解成本分别占 3.18%、2.23%，成品检测为 0。售价 200 元与总成本形成 60.22 元期望利润空间。

5.3.3 问题三的模型检验与结果分析

为验证模型求解的正确性与可靠性，本文从近优策略比较、解析闭式、Monte Carlo 独立模拟、极端情形与灵敏度五个维度进行交叉检验。

(1) 近优策略比较

仅报告单一最优解不足以判断方案是否稳健，因此进一步比较期望利润最高的十种策略，如图 18所示。



图 18 最优检测策略 Top 10 排序及决策结构

最优策略利润 60.22 元，次优 60.04 元，仅差 0.18 元；第二至第四的策略主要在 C_3 、 C_6 、 C_8 中取消一次零件检测，其余决策不变。说明最优解附近存在收益接近的可替代方案， C_3 、 C_6 、 C_8 检测处于成本—风险临界区域；半成品检测、拆解与返件拆解在高收益策略中高度一致，构成稳定骨架。实施时可锁定稳定骨架，在 C_3 、 C_6 、 C_8 检测安排上小范围调整。

(2) 解析闭式与树形 DP 交叉验证

将 BOM 分层递推的闭式解析结果与精确标量求解、树形 DP 三种独立实现分别比对：手算闭式与精确求解的利润差为 0，树形 DP 与扁平全枚举得到的最优策略完全一致（均为 1111111111111101），利润差亦为 0，成本分解之和与标量总成本的偏差约为 2.8×10^{-14} 。三种独立实现得到相同结果，说明状态压缩递推模型在数学上自洽、实现

正确。

(2) Monte Carlo 独立仿真验证

对最优策略分别以 20 万笔向量化模拟与 1 万笔逐事件模拟估计单位期望利润。两种模拟的理论期望利润 60.22 元均落入 95% 置信区间（向量化 [60.07, 60.29]，逐事件 [59.62, 60.62]），且各事件次数与理论期望绝对误差均小于 0.01，表明仿真与解析模型在总利润与内部流程层面均吻合。收敛过程如图 19 所示。

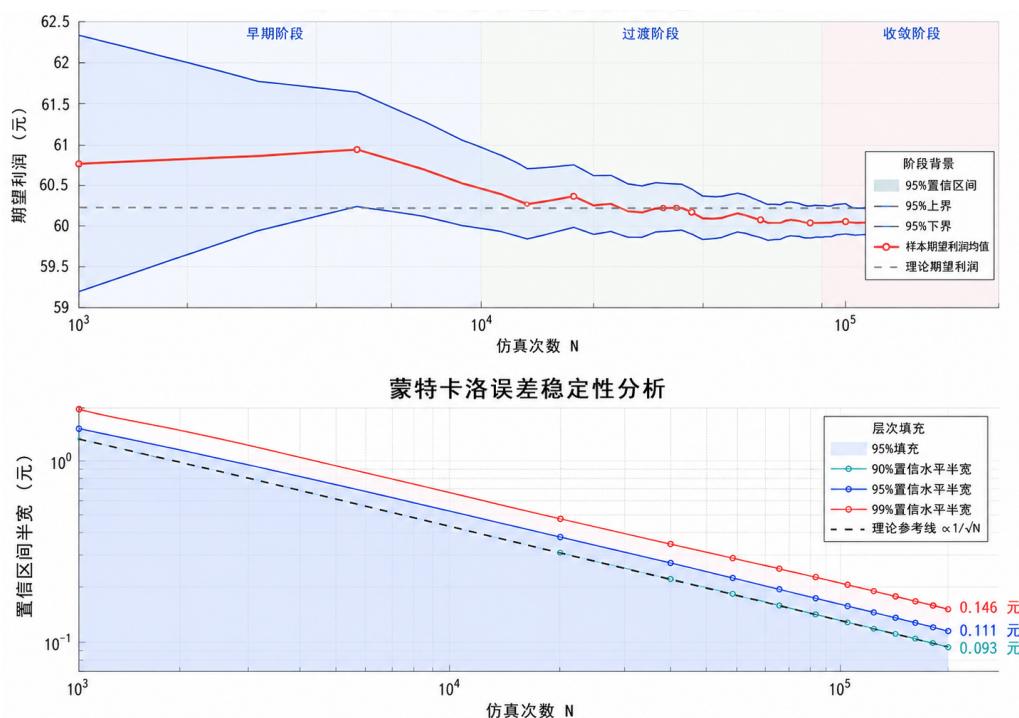


图 19 Monte Carlo 仿真期望利润收敛过程

随样本量增加，样本平均利润逐步贴近理论基准 60.22 元，置信区间同步收窄，说明偏差主要来自有限样本波动而非模型实现错误。

(4) 极端情形检验

构造四类极端参数场景验证模型经济学直觉：次品率全为 0 时退化为全不检测、全不拆解；调换损失极大时开启成品检测 ($y_F = 1$)；拆解成本极大时关闭全部拆解 ($z = 0$)；零件检测成本极大时关闭全部零件检测 ($x_{Ci} = 0$)。四种场景策略选择均与理论预期一致，验证了模型逻辑正确性。

(5) 灵敏度分析

对关键参数进行临界值扫描，得到最优策略的切换阈值。零配件检测成本 b_{Ci} (12 元) 低于阈值 2.22 元，表明用一次检测剔除次品所避免的后续装配与调换损失，大于检测投入本身，故前端零件检测经济合理；半成品检测成本 $b_{Hj} = 4$ 元低于阈值 7.16 元，在更靠后的工序节点，一次检测可同时阻断多个零件缺陷的下游传播，边际收益更高，故半成品检测同样合理。调换损失 $L = 40$ 元低于阈值 60 元、成品次品率 $p_F = 0.1$ 低于

阈值 0.15，表明成品层缺陷流出引发的售后代价尚有限，出厂检测费用大于其可避免的期望损失，故成品不检测；一旦 L 或 p_F 越过阈值，售后风险反超检测成本，策略切换为成品出厂检测。半成品拆解成本 $d_{Hj} = 6$ 元与成品拆解成本 $d_F = 10$ 元均远低于对应阈值，回收件价值大于拆解投入，故各拆解动作稳健。上述结论与最优策略一致，验证了解的正确性与稳健性。

为更直观地刻画决策边界与参数影响，分别绘制参数相图与关键参数影响气泡图。

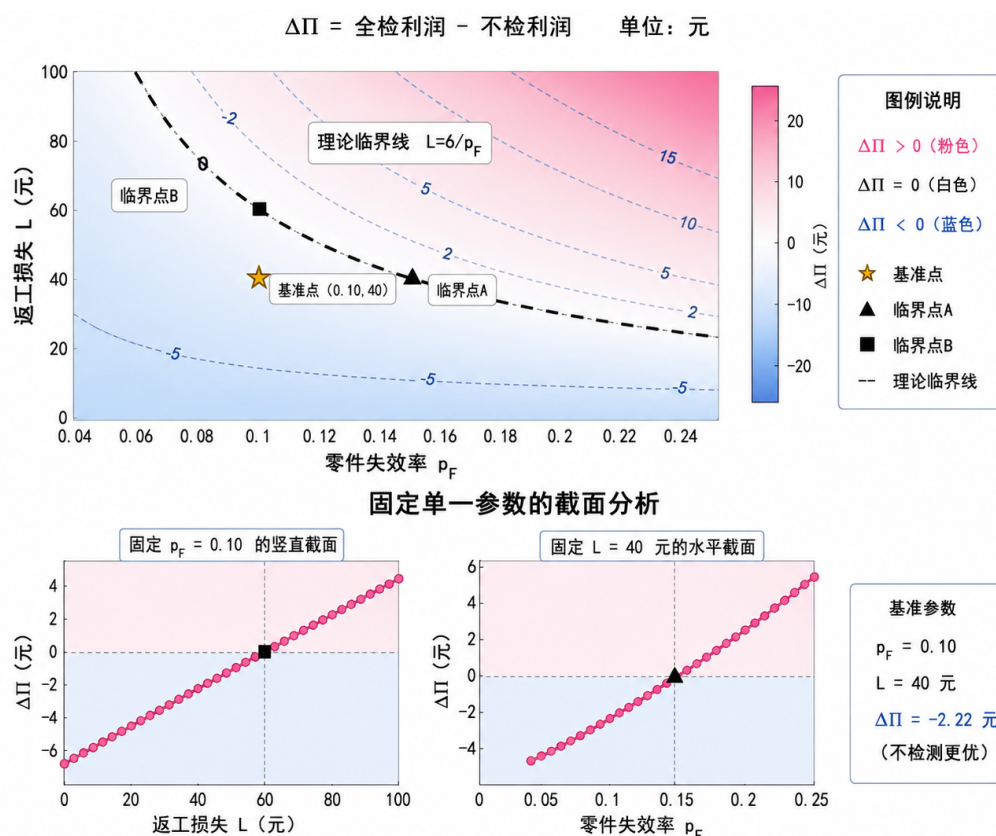


图 20 检测策略利润差参数相图

图 20 以关键参数的联合变化为坐标，用利润差刻画决策边界，直观揭示了最优策略随参数漂移而切换的临界位置。在问题三的全局重新优化下，主要切换点分别为调换损失约 47.68 元、成品工序不合格率约 0.1248、半成品检测成本约 5.30 元，三者均早于单参数局部理论给出的阈值，即调换损失 60 元与成品不合格率 0.15。出现这一提前现象的原因在于，全局搜索允许检测、拆解等多个决策同时变化，当某一参数尚未到达单点盈亏平衡时，另一些决策的协同调整已使其他策略的整体期望利润率先超过当前基准策略，从而触发策略切换。这一结果表明，仅依据单参数灵敏度判断策略稳定性会高估其稳健范围，而联合相图能够更保守、更真实地刻画决策边界，为企业在参数波动下的策略选择提供了更可靠的依据。

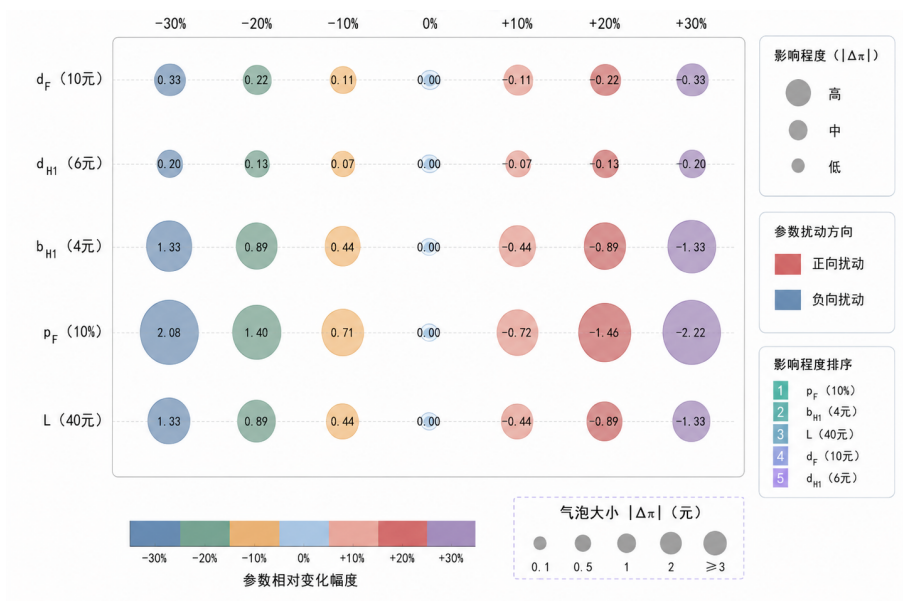


图 21 关键参数对最优期望利润的影响

图 21 以气泡图比较各关键参数的影响方向与幅度，气泡颜色区分成本增减、面积反映利润变化幅度。直接改变投入或失效传播概率的参数（材料单价、良品率、返工率）对利润影响显著，检测时间、能耗等参数影响较弱； C_3 、 C_6 、 C_8 检测费接近局部阈值，与次优策略利润接近相互印证。企业应优先监控高敏感参数，对接近切换阈值的检测环节保留动态调整能力。

5.3.4 问题三的结论

（1）问题三的最优生产方案为：八种零配件与三个半成品均检测、各半成品次品均拆解、成品不检测、用户退回次品拆解，对应期望成本 **139.78 元**、期望利润 **60.22 元**。此方案下，前端全面检测及早阻断质量风险，末端因三个半成品均已合格故不设检测反而更经济，各次品一律选择拆解以回收合格部件。

（2）该模型结果经解析闭式、蒙特卡洛模拟、极端情形与灵敏度检验均一致，求解可靠，可为同类多工序、多零部件装配系统提供量化决策依据。

5.4 问题四的模型建立与求解

5.4.1 问题四的模型建立

问题四在前两问的基础上，将次品率由已知改为未知。因此需要在生产决策模型中引入统计不确定性。

1. 次品率抽样推断模型

对每个质量参数，记真实次品率为 p_j ，独立抽样 n_j 件，检测结果 $X_{jt} \stackrel{\text{i.i.d.}}{\sim} B(p_j)$ ，累计次品数 $K_j = \sum_{t=1}^{n_j} X_{jt}$ 。则：

$$K_j \sim B(n_j, p_j), \quad \hat{p}_j = \frac{k_j}{n_j}. \quad (5.24)$$

其中 $\hat{p}_j$ 为次品率的点估计值，可作为基准参数代入生产决策。但抽样估计存在不确定性，为控制决策风险，本文在风险水平 γ 下计算置信上界 $U_j(\gamma)$ 。

2. 次品率置信上界模型

沿用问题一的 **e-process** 序贯检验框架。由于每个质量参数均在证据充足即停止的规则下得到，其样本量 n_j 本身即为随机停止时间，故对每个参数沿用问题一的证据分数 $E_{j,t}^-(p_0)$ ，将候选参数 p_0 逐一作为原假设边界进行反演，得到对应的置信上界：

$$U_j^s(\gamma) = \inf\{p_0 \in (0, 1] : E_{j,t}^-(p_0) \geq \frac{1}{1-\gamma}\}. \quad (5.25)$$

3. 多次品率同时置信包络模型

问题二每种情形含 $d = 3$ 个不确定参数，问题三实例含 $d = 12$ 个。为保证整个生产系统的所有质量参数同时落在上界内，采用 **Bonferroni** 将总体风险 $\alpha = 1 - \gamma$ 分配到各参数，令 $\alpha_j = \alpha/d$ ，构造系统级同时置信集合：

$$U_\gamma = \{p : 0 \leq p_j \leq U_j^s(\gamma), j = 1, \dots, d\}, \quad U_j^s(\gamma) = U_j(1 - \alpha/d). \quad (5.26)$$

该集合保证所有参数同时落在界内的概率不低于 γ ，且不要求各参数独立。

4. 最坏次品率下的生产决策重优化模型

将置信上界向量作为质量风险包络，代入前两问的精确评价器，在置信集合 U_γ 上对所有候选策略重新优化。

沿用问题二、问题三的决策变量与精确评价器。在给定质量参数向量 p 下，问题二解 **Bellman** 线性方程得到长期期望成本 $V_\pi(p)$ ，问题三由 **BOM** 分层递推得到根节点期望成本 $V_F(\pi; p)$ 。利润均记为：

$$\Pi_q(\pi; p) = S - V_q(\pi; p), \quad q = 2, 3. \quad (5.27)$$

第四问在置信水平 γ 下的核心目标函数为：

$$\pi_q^*(\gamma) = \arg \max_{\pi \in \Pi_q} \min_{p \in U_\gamma} [S - V_q(\pi; p)], \quad q = 2, 3. \quad (5.28)$$

约束条件为：

$$\text{s.t.} \quad \begin{cases} \pi \in \{0, 1\}^d, \\ z = 0 \Rightarrow r_1 = r_2 = 0, \\ p \in U_\gamma, \\ \sup_{p \in U_\gamma} \rho(Q_\pi(p)) < 1. \end{cases} \quad (5.29)$$

目标函数中的内层取最小值，即在最坏质量参数下评估每个策略，是出于稳健性考虑：真实次品率未知，而生产决策一旦确定便难以中途调整，若仅以点估计优化，一旦真实质量恶化，原策略的利润将大幅下滑。因此在置信集合上考察每个策略的最坏表现，可保证所选策略无论真实次品率落在 U_γ 内何处，利润都不低于该最坏值。

在本题生产结构下，任一质量参数 p_j 增大只会使不合格品增多，从而抬升期望成本、压缩期望利润，而不会产生额外收益。因此固定策略的期望利润 $\Pi_q(\pi; p)$ 关于各缺陷概率具有单调非增性，即对 $p \preceq p'$ 有 $\Pi_q(\pi; p) \geq \Pi_q(\pi; p')$ 。由此置信集合 U_γ 上的最坏点必落在各参数同时取上界处，可将最坏情形搜索简化为单点代入：

$$\min_{p \in U_\gamma} \Pi_q(\pi; p) = \Pi_q(\pi; U_\gamma^u). \quad (5.30)$$

5. 稳健决策检验模型

对于最优策略，还需判断当前样本信息是否足以支撑稳定决策。

一方面计算最优一次优利润间隔。设最优、次优策略分别为 $\pi_{(1)}$ 、 $\pi_{(2)}$ ：

$$\Delta = \Pi(\pi_{(1)}) - \Pi(\pi_{(2)}), \quad \Delta_r = \frac{\Delta}{|\Pi(\pi_{(1)})|}. \quad (5.31)$$

若 Δ 很小，即使最优策略数学上唯一，实际采样稍有变化也可能更换策略。

另一方面采用与原抽样设计一致的 **bootstrap** 方法，固定样本按 $K^* \sim \text{Binomial}(n_j, \hat{p}_j)$ 重新抽样，序贯样本必须逐件生成 **Bernoulli** 流并复现问题一的停止规则。第 b 次重抽样与重优化得到 π_b^* ，则策略稳定率为：

$$R_\pi = \frac{1}{B} \sum_{b=1}^B I(\pi_b^* = \pi), \quad \text{SE}(R_\pi) = \sqrt{\frac{R_\pi(1 - R_\pi)}{B}}. \quad (5.32)$$

当 R_π 较高且 Δ 较大时，策略兼具经济优势与统计稳定性，可作为可靠决策采用；当 R_π 较低或多个策略频率接近时，表明当前样本信息尚不足以固定单一最优策略，应以继续抽样缩小不确定性为管理建议。

此外，为分离质量参数恶化造成的损失与采用稳健策略付出的额外成本，引入四个交叉评价量。记 π_N 为名义最优策略、 π_R 为稳健最优策略， $\hat{p}$ 为点估计向量， U 为同时上界向量：

$$P_a = \Pi(\pi_N; \hat{p}), \quad P_b = \Pi(\pi_R; \hat{p}), \quad P_c = \Pi(\pi_N; U), \quad P_d = \Pi(\pi_R; U), \quad (5.33)$$

$$P_e = P_a - P_b, \quad P_f = P_d - P_c, \quad P_g = P_a - P_c, \quad P_h = P_d. \quad (5.34)$$

其中 P_e 衡量名义状态下采用稳健策略少赚的利润， P_f 衡量不利情形下稳健策略相对名义策略多保护的利润， P_g 反映参数恶化本身造成的损失， P_h 给出当前不确定集合下的最坏利润保证。该分解只用于解释既有最优策略，不改变目标函数。

5.4.2 问题四的求解结果

为刻画抽样不确定性如何进入优化模型，绘制各质量参数的同时上置信界增量，如图 22 所示。90% 同时上界对应增量约 0.037 ~ 0.094，95% 提高到约 0.044 ~ 0.105，置信水平提升系统性扩大了质量参数的保守修正。

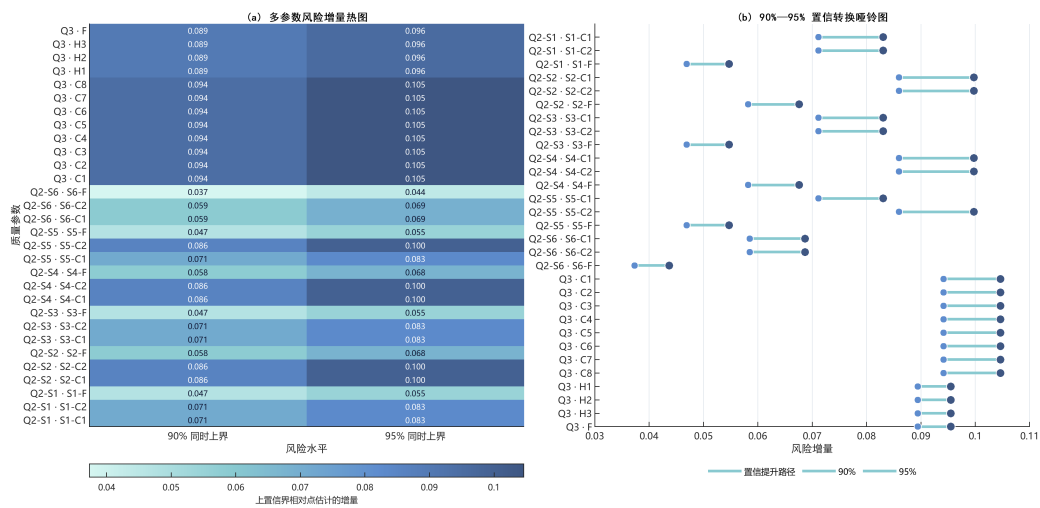


图 22 抽样不确定性的风险增量与置信水平效应

在 fixture 设计抽样场景下，对问题二六种情形与问题三实例分别在名义、90% 与 95% 稳健置信水平下重新优化，结果汇总于表 9（见附录）。

问题二六情形中四种（S1、S3、S5、S6）策略切换，方向均为增加一道检测；S2、S4 不变。问题三 95% 稳健下唯一变化为开启成品检测，利润由 60.22 元降至 36.75 元。企业以约 2.22 元/订单日常代价换取不利情形下约 2.26 元/订单保护，代价与收益对称。策略稳定率普遍超 0.93，S6 约 0.66 建议补抽样。

将三种风险层级组织为连续决策路径，如图 23 所示。

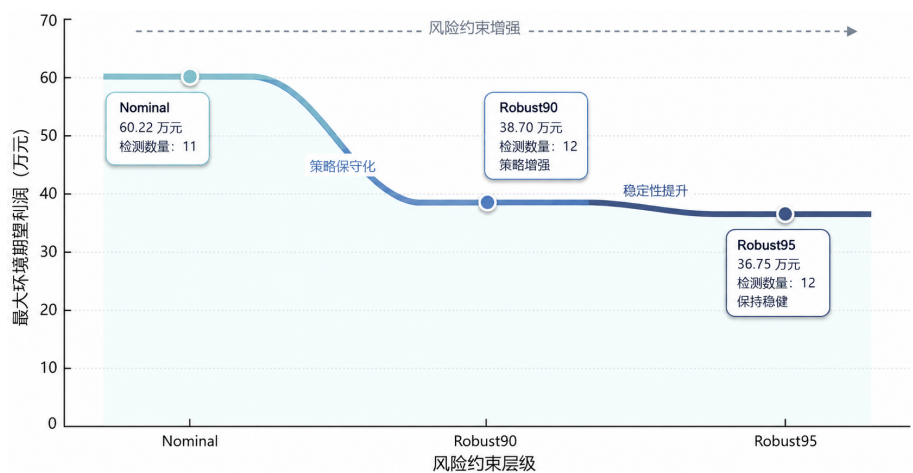


图 23 风险约束驱动的最优策略迁移路径

主要策略调整发生在名义向 90% 稳健转换阶段（检测由 11 道增至 12 道），90% 到 95% 则是在既有结构下继续增加安全裕度。

5.4.3 问题四的模型检验与结果分析

问题四的检验覆盖统计层、生产层与稳健层三个环节，共 17 项验收全部通过。

(1) 统计层检验：固定样本 Clopper-Pearson 上界 90%/95% 区间覆盖率均在 0.907 ~ 1.0；序贯 e-process 反演上界覆盖率不低于 0.998，均满足置信保证。Bonferroni 上界与全局诊断一致（误差 10^{-11} ），系统级同时置信集合构造正确。

(2) 生产层检验：问题二 48 个策略格点的 Bellman 解与闭式解最大误差 4.3×10^{-14} ；问题三七项成本分量手工核算与程序输出一致，总成本 139.78 元，误差 2.8×10^{-14} ，评价器精度无损。

(3) 稳健层检验：利润单调性检验显示各参数利润最大上升违反均为 0，确认利润关于缺陷概率单调非增；问题二 8 个角点与问题三 4096 个角点的实际最坏利润均与上界角点一致，确认最坏点为各参数同时取上界。相图检验确认切换阈值 $p_F^* \approx 0.15$ ，与解析临界 6/40 吻合。

为识别策略切换在参数空间的位置，比较名义与 95% 稳健策略的真实决策边界，如图 24 所示。95% 稳健边界整体位于名义边界右侧，边界移动量随调换损失 L 降低而增大，表明风险约束对临界次品率的影响在低 L 区间更显著。

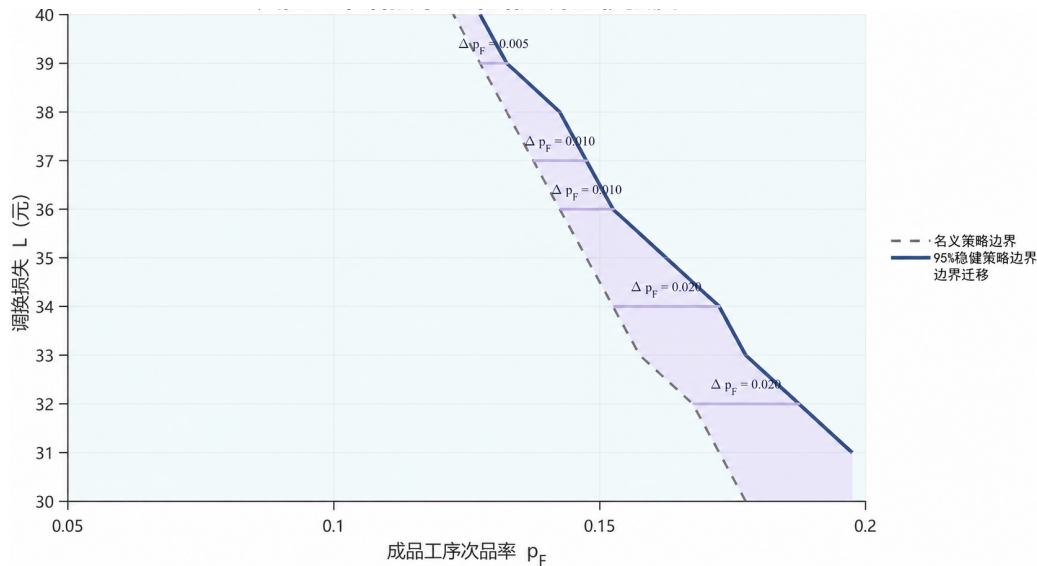


图 24 风险约束增强下的策略边界迁移强度

综上，统计层的覆盖保证、生产层的求解精度、稳健层的单调性与角点性质均经数值验证，第四问模型在 fixture 设计抽样场景下结果可靠。

为进一步检验策略选择的稳定性，以 $B = 512$ 次自助法重复抽样比较各情形的策略稳定率，如图 25 所示。多数算例在提高风险层级后稳定率明显增加，Q3 由名义的 10.9%

跃升至 91.8%，表明稳健重优化显著扩大了策略稳定裕度；蒙特卡洛标准误约 0 ~ 2.2 个百分点，远小于主要稳定率差异。

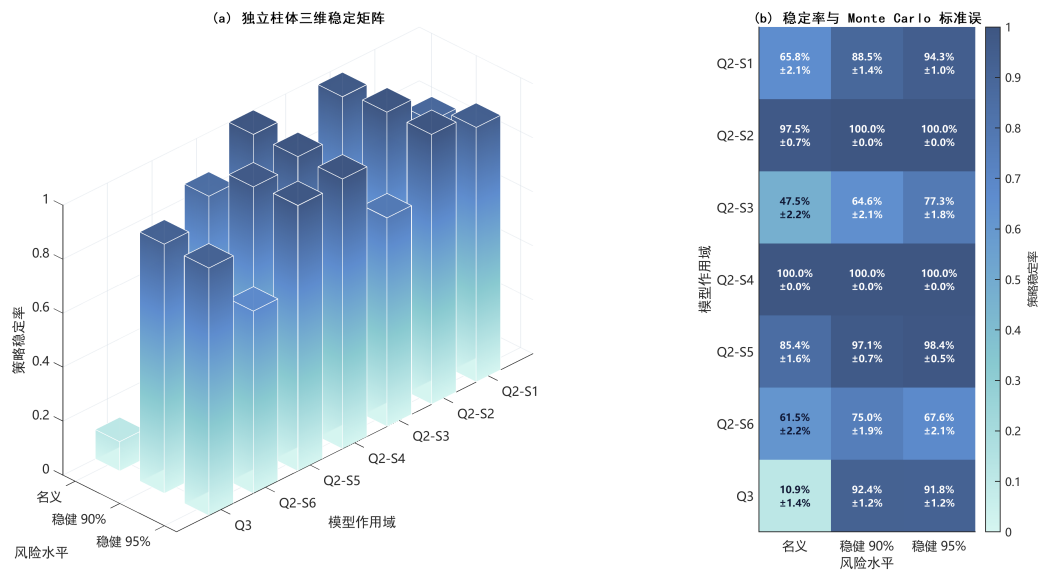


图 25 自助法策略稳定性矩阵及蒙特卡洛标准误

5.4.4 问题四的结论

(1) 对于问题二：95% 稳健水平下，六种情形中四种（S1、S3、S5、S6）最优策略切换，方向均为**增加一道检测**；S2、S4 不变。策略稳定率普遍**超过 0.93**（S2、S4 达 1.0），S6 名义水平约 0.66，建议补充抽样。

(2) 对于问题三：95% 稳健水平下唯一变化为**开启成品检测**，期望成本升至 **163.25 元**、利润降至 **36.75 元**。企业以约 **2.22 元/订单**的日常代价换取不利情形下约 **2.26 元/订单**的额外保护，**代价与收益基本对称**。

综上，第四问共 **17 项验收全部通过**，表明稳健优化只在决策接近经济临界点时触发策略调整，可为企业在抽样信息有限条件下的稳健生产决策提供量化依据。

六、模型的评价

6.1 模型的优点

- 模型层次清晰、层层递进，四问共享统一的评价框架，逻辑连贯、可读性强。
- 抽样方案兼顾判定可靠性与检测经济性。
- 求解采用精确枚举与闭式递推，结果为全局最优，稳定性好。
- 稳健优化具有可解释的结构性质，最坏情形简化有经济逻辑支撑，适用性强。
- 决策建议配有经济价值分解与灵敏度指标，管理可读性好，便于企业落地。

6.2 模型的缺点

- 策略空间随决策位数指数增长，对更大规模的装配系统可扩展性有限。
- 序贯抽样在边界情形的稳定性检验存在截尾问题，统计解释力受限。

七、 AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、代码调试，详细使用情况见支撑材料。

参考文献

- [1] 李大潜. 从数学建模到问题驱动的应用数学 [J]. 数学建模及其应用, 2014, 3(1).
- [2] 丁源. 面向小样本约束的成败型产品验收抽样方案对比分析 [J]. 舰船电子对抗, 2026, 49(03): 45-48. DOI: 10.16426/j.cnki.jcdzdk.2026.03.008.
- [3] 申绪润, 戚宗锋, 汪连栋, 王国玉, 王雪松. 二项分布未知参数的序贯验后加权检验方法 [J]. 无线电工程, 2003(10): 27-30.
- [4] Bertsimas D, Sim M. The Price of Robustness[J]. *Operations Research*, 2004, 52(1): 35-53.
- [5] 王国鸿, 宁汝新. 对制造业物料清单的研究 [J]. 北京理工大学学报（自然科学版）, 2001(2): 196-200.
- [6] 司守奎, 孙玺菁. 数学建模算法与应用 [M]. 北京: 国防工业出版社, 2011.
- [7] 薛丽, 何桢. 二项分布下 VSI EWMA 控制图的经济设计 [J]. 系统工程学报, 2020, 35(5): 711-720.
- [8] 梁斌斌, 邵荃, 蔡鑫宇, 朱培, 张生润, 李炜. 低空航行风险感知、评估与控制方法研究综述 [J]. 北京理工大学学报（自然科学版）, 2026, 46(5): 451-469. DOI: 10.15918/j.tbit1001-0645.2025.166.

附录 A 支撑材料文件清单

文件名	功能描述
q1_model.py	问题一：e-process 序贯抽样模型与精确二项检验
q2_model.py	问题二：Bellman 生产循环状态方程与 64 策略枚举
q3_model_refactored.py	问题三：BOM 分层递推与 2^{16} 策略全枚举
Q4_main_model.py	问题四：Bonferroni 校正与系统置信集稳健重优化
generate_all_figures.m	问题一：五张论文图的一键生成入口
q2_generate_all_figures_v5_2.m	问题二：v5.2 版六图统一生成脚本
q3_generate_figures.m	问题三：图 2–图 6 严格版独立生成函数
generate_q4_figures.m	问题四：稳健优化论文图表一键生成入口
Q2_color.m	第二/三问共用自然色板定义函数
Q2_style.m	第二/三问共用字体/字号/线宽样式函数

附录 B 问题二六种情形的稳健重优化结果

表 9 问题二六种情形的稳健重优化结果（单位：元）

情形	置信水平	最优策略	期望成本	期望利润
S1	名义	1001/100111	37.16	18.84
	90%	1101/110111	41.50	14.50
	95%	1101/110111	42.16	13.84
S2	名义	1101/110111	44.00	12.00
	90%	1101/110111	49.73	6.27
	95%	1101/110111	50.77	5.23
S3	名义	1011/101111	39.53	16.47
	90%	1111/111111	43.99	12.01
	95%	1111/111111	44.61	11.39
S4	名义	1111/111111	41.25	14.75
	90%	1111/111111	46.14	9.86
	95%	1111/111111	47.02	8.98
S5	名义	0101/010111	41.04	14.96
	90%	0111/011111	47.89	8.11
	95%	0111/011111	49.06	6.94
S6	名义	0000/000000	34.32	21.68
	90%	1000/100000	40.06	15.94
	95%	1000/100000	41.09	14.91

附录 C 问题一代码

```
1 class EProcessConfig:
2     """e-process 配置。prior_a=prior_b=1 即主模型"均匀混合"."
3
4     p0: float = DEFAULT_P0
5     accept_confidence: float = DEFAULT_ACCEPT_CONFIDENCE
6     reject_confidence: float = DEFAULT_REJECT_CONFIDENCE
7     prior_a: float = 1.0
8     prior_b: float = 1.0
9     mixture_name: str = "uniform"
10
11     @property
12     def alpha_accept(self) -> float:
13         return 1.0 - self.accept_confidence
14
15     @property
16     def alpha_reject(self) -> float:
17         return 1.0 - self.reject_confidence
18
19     @property
20     def log_accept_threshold(self) -> float:
21         return math.log(1.0 / self.alpha_accept)
22
23     @property
24     def log_reject_threshold(self) -> float:
25         return math.log(1.0 / self.alpha_reject)
26
27     def validate(self) -> None:
28         if not (0.0 < self.p0 < 1.0):
29             raise ValueError("p0 必须位于 (0,1) 内。")
30         if not (0.0 < self.accept_confidence < 1.0):
31             raise ValueError("accept_confidence 必须位于 (0,1)
内。")
```

```

32         if not (0.0 < self.reject_confidence < 1.0):
33             raise ValueError("reject_confidence 必须位于 (0,1)
        内。")
34         if self.prior_a <= 0.0 or self.prior_b <= 0.0:
35             raise ValueError("Beta 混合参数 prior_a, prior_b
        必须为正。")
36
37
38 @dataclass
39 def _log_lower_incomplete_beta(a: float, b: float, x: float)
    -> float:
40     """log B_x(a,b), 使用 regularized betainc + betaln。"""
41     reg = float(betainc(a, b, x))
42     if reg <= 0.0:
43         return -math.inf
44     return float(betaln(a, b)) + math.log(reg)
45
46
47 def _log_upper_incomplete_beta(a: float, b: float, x: float)
    -> float:
48     """log [B(a,b)-B_x(a,b)], 使用 betaincc 避免 1-betainc 的
        消减误差。"""
49     reg_c = float(betaincc(a, b, x))
50     if reg_c <= 0.0:
51         return -math.inf
52     return float(betaln(a, b)) + math.log(reg_c)
53
54
55 def log_e_minus(n: int, k: int, config: EProcessConfig) ->
    float:
56     config.validate()
57     if not (0 <= k <= n):
58         raise ValueError("必须满足 0 <= k <= n。")
59
60     p0 = config.p0

```

```

61     a0, b0 = config.prior_a, config.prior_b
62
63     log_num = _log_lower_incomplete_beta(k + a0, n - k + b0,
64     p0)
65     log_mix_norm = _log_lower_incomplete_beta(a0, b0, p0)
66     log_l0 = k * math.log(p0) + (n - k) * math.log1p(-p0)
67     return log_num - log_mix_norm - log_l0
68
69 def log_e_plus(n: int, k: int, config: EProcessConfig) ->
70     float:
71     config.validate()
72     if not (0 <= k <= n):
73         raise ValueError("必须满足 0 <= k <= n。")
74
75     p0 = config.p0
76     a0, b0 = config.prior_a, config.prior_b
77
78     log_num = _log_upper_incomplete_beta(k + a0, n - k + b0,
79     p0)
80     log_mix_norm = _log_upper_incomplete_beta(a0, b0, p0)
81     log_l0 = k * math.log(p0) + (n - k) * math.log1p(-p0)
82     return log_num - log_mix_norm - log_l0
83
84 def sequential_decision(n: int, k: int, config: EProcessConfig
85     ) -> str:
86
87     lm = log_e_minus(n, k, config)
88     lp = log_e_plus(n, k, config)
89
90     hit_accept = lm >= config.log_accept_threshold
91     hit_reject = lp >= config.log_reject_threshold
92
93     if hit_accept and hit_reject:
94         raise RuntimeError(f"状态 (n={n}, k={k}) 同时触发接收

```

```

与拒收阈值。")
92     if hit_accept:
93         return "accept"
94     if hit_reject:
95         return "reject"
96     return "continue"
97
98
99 def _accept_boundary_binary(n: int, config: EProcessConfig) ->
Optional[int]:
100     thr = config.log_accept_threshold
101 def main() -> None:
102     parser = argparse.ArgumentParser(
103         description="2024B 问题一：精确二项 + 混合似然比 e-
process 序贯抽样"
104     )
105     parser.add_argument(
106         "--max-n",
107         type=int,
108         default=DEFAULT_MAX_N,
109         help=f"主递推/主边界最大样本量，默认 {DEFAULT_MAX_N}",
110     )
111     parser.add_argument(
112         "--fixed-max-n",
113         type=int,
114         default=DEFAULT_FIXED_MAX_N,
115         help=f"固定样本边界输出上限，默认 {DEFAULT_FIXED_MAX_N}
",
116     )
117     parser.add_argument(
118         "--output-dir",
119         type=str,
120         default=" ../结果输出",
121         help=(
122             "输出目录，默认相对脚本所在目录的 ../结果输出"

```

```
123         "PyCharm 中直接运行即可把所有 CSV 与检验报告写入"  
124         "求解代码/第一问/结果输出/, 供 MATLAB 绘图复用"  
125     ),  
126 )  
127 parser.add_argument(  
128     "--sample-csv",  
129     type=str,
```

Listing 1: 问题一核心模型节选

附录 D 问题二代码

```
1 class Policy:
2     x1: int
3     x2: int
4     y: int
5     z: int
6     r1: int
7     r2: int
8
9     def validate(self) -> None:
10         vals = (self.x1, self.x2, self.y, self.z, self.r1,
11 self.r2)
12         if any(v not in (0, 1) for v in vals):
13             raise ValueError(f"策略变量必须均为0/1，当前={vals
14 }")
15
16     def canonical(self) -> "Policy":
17         """z=0 时 r1,r2 无实际意义，统一规范化为0"""
18         if self.z == 0:
19             return Policy(self.x1, self.x2, self.y, 0, 0, 0)
20         return self
21
22     @property
23     def base4(self) -> Tuple[int, int, int, int]:
24         return self.x1, self.x2, self.y, self.z
25
26     @property
27     def code4(self) -> str:
28         return f"{self.x1}{self.x2}{self.y}{self.z}"
29
30     @property
31     def code6(self) -> str:
32         return f"{self.x1}{self.x2}{self.y}{self.z}{self.r1}{
33 self.r2}"
```

```

31
32
33 @dataclass
34 class PolicyEvaluation:
35     scenario_id: Union[int, str]
36     policy: Policy
37     feasible: bool
38     expected_cost: float
39     expected_profit: float
40     spectral_radius: float
41     condition_number: float
42     bellman_residual_inf: float
43     absorption_probability: float
44     reachable_states: Tuple[Union[str, Tuple[str, str]], ...]
45     cost_breakdown: Dict[str, float]
46     event_counts: Dict[str, float]
47     reason: str = ""
48
49
50 # 六种情形参数
51 SCENARIOS: List[ScenarioParams] = [
52     ScenarioParams(1, 0.10, 4, 2, 0.10, 18, 3, 0.10, 6, 3, 56,
53     6, 5),
54     ScenarioParams(2, 0.20, 4, 2, 0.20, 18, 3, 0.20, 6, 3, 56,
55     6, 5),
56     ScenarioParams(3, 0.10, 4, 2, 0.10, 18, 3, 0.10, 6, 3, 56,
57     30, 5),
58     ScenarioParams(4, 0.20, 4, 1, 0.20, 18, 1, 0.20, 6, 2, 56,
59     30, 5),
60     ScenarioParams(5, 0.10, 4, 8, 0.20, 18, 1, 0.10, 6, 2, 56,
61     10, 5),
62     ScenarioParams(6, 0.05, 4, 2, 0.05, 18, 3, 0.05, 6, 3, 56,
63     10, 40),
64 ]

```



```

60 # 外部交叉检查结果
61 REFERENCE_BEST = {
62     1: ("1001", 18.8411),
63     2: ("1101", 12.0000),
64     3: ("1011", 16.4744),
65     4: ("1111", 14.7500),
66     5: ("0101", 14.9633),
67     6: ("0000", 21.6787),
68 }
69
70
71 # 数据处理与基础概率
72 def params_to_dataframe(params_list: Sequence[ScenarioParams])
    -> pd.DataFrame:
73 def build_markov_model(
74     p: ScenarioParams,
75     policy: Policy,
76 ) -> Tuple[
77     List[Union[str, Tuple[str, str]]],
78     np.ndarray,
79     Dict[str, np.ndarray],
80     Dict[str, np.ndarray],
81 ]:
82
83     p.validate()
84     policy = policy.canonical()
85     policy.validate()
86
87     states: List[Union[str, Tuple[str, str]]] = [STATE_N] +
    PAIR_STATES
88     index = {s: i for i, s in enumerate(states)}
89     n = len(states)
90
91     Q = np.zeros((n, n), dtype=float)
92     cost_reward = {k: np.zeros(n, dtype=float) for k in

```

```

COST_CATEGORIES}
93     count_reward = {k: np.zeros(n, dtype=float) for k in
COUNT_CATEGORIES}
94
95     # 首次采购/检测，然后立即完成一轮装配
96     prep_cost, prep_count = _initial_preparation_reward(p,
policy)
97     for k, v in prep_cost.items():
98         cost_reward[k][0] += v
99     for k, v in prep_count.items():
100         count_reward[k][0] += v
101
102     cost_reward["assembly"][0] += p.assembly_cost
103     count_reward["assemblies"][0] += 1.0
104     if policy.y == 1:
105         cost_reward["final_inspection"][0] += p.bf
106         count_reward["final_inspections"][0] += 1.0
107
108     for prob1, s1 in initial_latent_branches(p.p1, policy.x1):
109         for prob2, s2 in initial_latent_branches(p.p2, policy.
x2):
110             branch_prob = prob1 * prob2
111             state_pair = (s1, s2)
112             fail_prob = product_failure_probability(state_pair
, p)
113
114             if policy.y == 0:
115                 cost_reward["replacement"][0] += branch_prob *
fail_prob * p.replacement_loss
116                 count_reward["replacements"][0] += branch_prob
* fail_prob
117
118             if policy.z == 1:
119                 cost_reward["disassembly"][0] += branch_prob *
fail_prob * p.disassembly_cost

```

```

120         count_reward["disassemblies"][0] +=
branch_prob * fail_prob
121         Q[0, index[state_pair]] += branch_prob *
fail_prob
122     else:
123         Q[0, index[STATE_N]] += branch_prob *
fail_prob
124
125     for state_pair in PAIR_STATES:
126         row = index[state_pair]
127
128         new1, c1, n1 = recovery_process_component(
129             state_pair[0], policy.r1, p.p1, p.a1, p.b1
130         )
131         new2, c2, n2_raw = recovery_process_component(
132             state_pair[1], policy.r2, p.p2, p.a2, p.b2
133         )
134         n2 = remap_component_count_keys(n2_raw, component=2)
135
136         for k, v in c1.items():
137             cost_reward[k][row] += v
138         for k, v in c2.items():
139             cost_reward[k][row] += v
140         for k, v in n1.items():
141             count_reward[k][row] += v
142         for k, v in n2.items():
143             count_reward[k][row] += v
144
145         processed_pair = (new1, new2)
146
147         cost_reward["assembly"][row] += p.assembly_cost
148         count_reward["assemblies"][row] += 1.0
149         if policy.y == 1:
150             cost_reward["final_inspection"][row] += p.bf
151             count_reward["final_inspections"][row] += 1.0

```

```

152
153     fail_prob = product_failure_probability(processed_pair
, p)
154
155     if policy.y == 0:
156         cost_reward["replacement"][row] += fail_prob * p.
replacement_loss
157         count_reward["replacements"][row] += fail_prob
158
159     if policy.z == 1:
160         cost_reward["disassembly"][row] += fail_prob * p.
disassembly_cost
161         count_reward["disassemblies"][row] += fail_prob
162         Q[row, index[processed_pair]] += fail_prob
163     else:
164         Q[row, index[STATE_N]] += fail_prob
165
166     return states, Q, cost_reward, count_reward
167
168
169 def evaluate_policy(p: ScenarioParams, policy: Policy) ->
PolicyEvaluation:
170     policy = policy.canonical()
171     states, Q, cost_reward, count_reward = build_markov_model(
p, policy)
172
173     # 基本概率合法性检查
174     row_sums = Q.sum(axis=1)
175     if np.any(row_sums > 1.0 + 1e-10) or np.any(Q < -1e-14):
176         return PolicyEvaluation(
177             p.scenario_id, policy, False, math.inf, -math.inf,
178             math.inf, math.inf, math.inf, 0.0, tuple(), {},
179             {},
180             reason="Q矩阵存在非法概率（行和>1或负概率）"
)

```

```

181
182     reachable = reachable_indices(Q, start=0)
183     Qr = Q[np.ix_(reachable, reachable)]
184
185     if Qr.size == 0:
186         rho = 0.0
187     else:
188         eigvals = np.linalg.eigvals(Qr)
189         rho = float(np.max(np.abs(eigvals))) if len(eigvals)
190     else 0.0
191
192     reachable_state_names = tuple(states[i] for i in reachable
193 )
194
195     if rho >= 1.0 - RHO_TOL:
196         return PolicyEvaluation(
197             p.scenario_id, policy, False, math.inf, -math.inf,
198             rho, math.inf, math.inf, 0.0,
199             reachable_state_names, {}, {},
200             reason="暂态矩阵谱半径>=1，存在永久循环/期望成本发
201             散"
202         )
203
204     A = np.eye(len(reachable)) - Qr

```

Listing 2: 问题二核心模型节选

附录 E 问题三代码

```
1 class Q3Parameters:
2
3     p_comp: List[float]
4     purchase_comp: List[float]
5     inspect_comp: List[float]
6
7     children: Dict[str, List[int]]
8     p_semi: Dict[str, float]
9     asm_semi: Dict[str, float]
10    inspect_semi: Dict[str, float]
11    disasm_semi: Dict[str, float]
12
13    p_final: float
14    asm_final: float
15    inspect_final: float
16    disasm_final: float
17    sale_price: float
18    replacement_loss: float
19
20    def clone(self) -> "Q3Parameters":
21        return copy.deepcopy(self)
22
23
24 @dataclass
25 class NodeDesc:
26     C: float
27     q: float
28     known: bool
29     K: float
30     D: float
31     b: float
32
33
```

```

34 @dataclass
35 class VecNodeDesc:
36     C: np.ndarray
37     q: float
38     known: bool
39     K: np.ndarray
40     D: np.ndarray
41     b: float
42     inspect_category: str
43
44
45 @dataclass
46 class LocalCandidate:
47     name: str
48     comp_indices: Tuple[int, ...]
49     x_bits: Tuple[int, ...]
50     y: int
51     z: int
52     desc: NodeDesc
53
54
55 @dataclass
56 class SimItem:
57
58     node: str
59     good: bool
60     known: bool
61     children: Optional[List["SimItem"]] = None
62
63
64 # 表2参数与数据校验
65 def default_parameters() -> Q3Parameters:
66 def internal_desc(
67     ch_descs: Sequence[NodeDesc],
68     p_proc: float,

```

```

69     asm_cost: float,
70     inspect_cost: float,
71     disasm_cost: float,
72     y: int,
73     z: int,
74 ) -> NodeDesc:
75     B = asm_cost + sum(c.C for c in ch_descs)
76     g = (1.0 - p_proc) * math.prod(c.q for c in ch_descs)
77
78     if not (0.0 < g <= 1.0 + EPS):
79         raise ArithmeticError(f"节点一次成功概率 g={g} 非法。")
80
81     g = min(1.0, g)
82
83     if z == 0:
84         K = (B + inspect_cost) / g
85         D = K
86     else:
87         # 父节点失败后，若拆解，则对未知子件做Bayes条件更新并
88         # 认证
89         fail_prob = 1.0 - g
90         cert = 0.0
91         if fail_prob > EPS:
92             for c in ch_descs:
93                 if c.known:
94                     continue
95                 bad_post = (1.0 - c.q) / fail_prob
96                 bad_post = min(1.0, max(0.0, bad_post))
97                 q_post = 1.0 - bad_post
98                 cert += recovery_cost(c, q_post)
99
100         # 当全部输入已知合格，只剩工序自身失败；对工序失败不断
101         # 拆解重装
102         K_proc = (asm_cost + inspect_cost + p_proc *
103                 disasm_cost) / (1.0 - p_proc)

```



```

100         D = disasm_cost + cert + K_proc
101         K = B + inspect_cost + fail_prob * D
102
103         if y == 1:
104             return NodeDesc(C=K, q=1.0, known=True, K=K, D=D, b=
inspect_cost)
105         return NodeDesc(C=B, q=g, known=False, K=K, D=D, b=
inspect_cost)
106
107
108 def final_cost(
109     par: Q3Parameters,
110     h_descs: Sequence[NodeDesc],
111     y_f: int,
112     z_f: int,
113 ) -> float:
114     B = par.asm_final + sum(c.C for c in h_descs)
115     g = (1.0 - par.p_final) * math.prod(c.q for c in h_descs)
116     if not (0.0 < g <= 1.0 + EPS):
117         raise ArithmeticError(f"成品一次成功概率 g_F={g} 非
法。")
118     g = min(1.0, g)
119
120     if y_f == 1:
121         # 成品出厂检测：坏品在厂内截获，无市场调换损失
122         desc = internal_desc(
123             h_descs,
124             par.p_final,
125             par.asm_final,
126             par.inspect_final,
127             par.disasm_final,
128             y=1,
129             z=z_f,
130         )
131         return desc.C

```

```

132
133     # 不检测成品：失败会流入市场并产生调换损失
134     if z_f == 0:
135         # 每次失败整套报废，从头开始
136         return (B + (1.0 - g) * par.replacement_loss) / g
137
138     # 不检测 + 用户退回后拆解
139     fail_prob = 1.0 - g
140     cert = 0.0
141     if fail_prob > EPS:
142         for c in h_descs:
143             if c.known:
144                 continue
145             bad_post = (1.0 - c.q) / fail_prob
146             bad_post = min(1.0, max(0.0, bad_post))
147             q_post = 1.0 - bad_post
148             cert += recovery_cost(c, q_post)
149
150     # 三个半成品全部认证合格后，只剩最终装配工序自身随机失败
151     M_f = (
152         par.asm_final
153         + par.p_final * (par.replacement_loss + par.
disasm_final)
154     ) / (1.0 - par.p_final)
155
156     return B + fail_prob * (
157         par.replacement_loss + par.disasm_final + cert + M_f
158     )
159
160
161 def evaluate_policy(par: Q3Parameters, bits: Sequence[int]) ->
    Tuple[float, float]:
162     if len(bits) != 16 or any(b not in (0, 1) for b in bits):
163         raise ValueError("策略必须是长度16的0-1序列。")
164 def tree_dp_search(par: Q3Parameters, top_k: int = 10,

```

```

return_all: bool = False):
165     cand = {name: build_local_candidates(par, name) for name
in ("H1", "H2", "H3")}
166     assert len(cand["H1"]) == 32
167     assert len(cand["H2"]) == 32
168     assert len(cand["H3"]) == 16
169
170     rows = [] if return_all else None
171     best_rows: List[Dict] = []
172
173     for h1, h2, h3 in product(cand["H1"], cand["H2"], cand["H3
174     "]):
175         hs = [h1.desc, h2.desc, h3.desc]
176         for y_f, z_f in product((0, 1), repeat=2):
177             cost = final_cost(par, hs, y_f, z_f)
178             profit = par.sale_price - cost
179
180             xs = [0] * 8
181             for c in (h1, h2, h3):
182                 for idx, x in zip(c.comp_indices, c.x_bits):
183                     xs[idx] = x
184             bits = tuple(xs + [h1.y, h2.y, h3.y] + [h1.z, h2.z
, h3.z] + [y_f, z_f])
185             rec = {
186                 "policy_bits": bits_to_string(bits),
187                 "profit": float(profit),
188                 "cost": float(cost),
189                 "bits": bits,
190             }
191             if rows is not None:
192                 rows.append(rec)
193
194     best_rows.append(rec)

```

Listing 3: 问题三核心模型节选

附录 F 问题四代码

```
1 def save(df: pd.DataFrame, path: Path) -> None:
2     if df is None or df.empty:
3         return
4     path.parent.mkdir(parents=True, exist_ok=True)
5     df.to_csv(path, index=False, encoding="utf-8-sig")
6
7
8 def resolve_output_dir(arg: str) -> Path:
9     p = Path(arg)
10    if not p.is_absolute():
11        p = THIS_DIR / p
12 def run_q4_robust(
13     out: Path,
14     records,
15     full_flat: bool,
16     args,
17     rep: Reporter,
18     data_source: str,
19 ) -> tuple[dict, pd.DataFrame, pd.DataFrame, pd.DataFrame]:
20     """基于真实抽样记录的稳健重优化 + 单调性审计 + PoR 分解 +
21     Bootstrap。"""
22     rep.section("Q4 稳健重优化（90%/95% 同时置信上界）", "3/4"
23 )
24
25     rep.step("Q2 6 情形 × (nominal / 90% / 95%) 稳健重优化", "
26 3.1")
27     q2_int, q2_res, q2_top, q2_mono = solve_q4_q2(records,
28 topk=10, data_source=data_source)
29     if not q2_int.empty: save(q2_int, out / "q4_q2_intervals.
30 csv")
31     if not q2_res.empty: save(q2_res, out / "q4_q2_policy.csv"
32 )
33     if not q2_top.empty: save(q2_top, out / "q4_q2_topk.csv")
```

```

28     if not q2_mono.empty: save(q2_mono, out / "
validation_q2_monotonicity.csv")
29     if not q2_res.empty:
30         rep.info(f"已求解 {q2_res['scenario_id'].nunique()} 个
情形；策略切换 {int((q2_res.groupby('scenario_id')['policy4
'].nunique() > 1).sum()))} 个")
31         print_q4_q2_policy_table(rep, q2_res)
32
33     q3_available = all(
34         any(r.quality_id == qid for r in records)
35         for qid in [*[f"Q3-C{i}" for i in range(1, 9)], "Q3-H1
", "Q3-H2", "Q3-H3", "Q3-F"]
36     )
37     q3_res = pd.DataFrame()
38     q3_int = pd.DataFrame()
39     q3_top = pd.DataFrame()
40     q3_mono = pd.DataFrame()
41     if q3_available:
42         rep.step("Q3 (nominal / 90% / 95%) 稳健重优化", "3.2")
43         q3_int, q3_res, q3_top, q3_mono = solve_q4_q3(
44             records, topk=10, flat_cross_check=full_flat,
data_source=data_source,
45         )
46         save(q3_int, out / "q4_q3_intervals.csv")
47         save(q3_res, out / "q4_q3_policy.csv")
48         save(q3_top, out / "q4_q3_topk.csv")
49         save(q3_mono, out / "validation_q3_monotonicity.csv")
50         rep.info(f"Q3 65536 独立平坦核验全部通过: {(q3_res['
crosscheck_PASS'] == True).all()} if 'crosscheck_PASS' in
q3_res.columns else '-}")")
51         print_q4_q3_policy_table(rep, q3_res)
52
53         rep.step("Q3 单调性审计 (zero / upper / midpoint 三类
锚点 × 12 参数)", "3.3")
54         rep.info(f"全部 PASS: {bool(q3_mono['PASS'].all())};

```

```

共 {len(q3_mono)} 项 (= 2 conf × 3 anchor × 12 参数) ")
55
56     rep.step("★ Price of Robustness 分解 (Q2/Q3) ", "3.4")
57     por2 = robustness_decomposition_q2(records,
family_size=3, data_source=data_source)
58     save(por2, out / "q4_robustness_decomposition.csv")
59     por3 = robustness_decomposition_q3(records,
family_size=12, data_source=data_source)
60     # 把 Q3 单行追加到同一表
61     all_por = pd.concat([por2, por3], ignore_index=True,
sort=False)
62     save(all_por, out / "q4_robustness_decomposition.csv")
63     rep.info(f"Q2 {len(por2)} 情形 + Q3 1 例, 全部输出
P_NN/P_RN/P_NU/P_RU/PoR/RG/PDE/WG")
64
65     rep.step("★ 策略位差 smoke 测试", "3.5")
66     ddiff = decode_diff_smoke()
67     save(ddiff, out / "q4_decode_diff_smoke.csv")
68     n_total = len(ddiff)
69     n_diff = int((ddiff["n_changes"] > 0).sum()) if not
ddiff.empty else 0
70     n_no_change = int((ddiff["n_changes"] == 0).sum()) if
not ddiff.empty else 0
71     rep.info(f"共 {n_total} 行 nominal→robust 切换; 其中 {
n_diff} 行有位差、{n_no_change} 行 nominal=robust (正确情
况) ")
72
73     rep.step("★ Q2 角点最坏点核验 (d=3 → 8 角点) ", "3.6")
74     rc2 = robust_corner_check_q2(records, top_k=5)
75     save(rc2, out / "validation_robust_corner_check_q2.csv
")
76     rep.info(f"全部 PASS: {bool(rc2['PASS'].all())}; 共 {
len(rc2)} 行")
77
78     rep.step("★ Q3 角点最坏点核验 (d=12 → 4096 角点) ", "

```

```

3.7")
79     rc3 = robust_corner_check_q3(records, top_k=5)
80     save(rc3, out / "validation_robust_corner_check_q3.csv
")
81     rep.info(f"全部 PASS: {bool(rc3['PASS'].all())}; 共 {
len(rc3)} 行")
82
83     rep.step("信息充分性曲线 (n×{1,2,4,8,16} 设计投影)",
"3.8")
84     info_parts = []
85     i2 = information_sufficiency_q2(records)
86     if not i2.empty: info_parts.append(i2)
87     i3 = information_sufficiency_q3(records)
88     if not i3.empty: info_parts.append(i3)
89     if info_parts:
90         info_df = pd.concat(info_parts, ignore_index=True,
sort=False)
91         save(info_df, out / "q4_information_curve.csv")
92         if "mean_upper_minus_phat" in info_df.columns:
93             narrow = info_df[info_df["sample_multiplier"
== 16]]["mean_upper_minus_phat"].mean()
94             wide = info_df[info_df["sample_multiplier" ==
1]]["mean_upper_minus_phat"].mean()
95             if pd.notna(wide) and wide > 0:
96                 rep.info(f" 设计样本 ×16 相比 ×1, 区间宽
度平均收缩 {(1 - narrow/wide) * 100:.1f}%")
97
98     # 合并区间表 (去重每 quality_id 仅保留一次)
99     intervals_combined = [x for x in [q2_int, q3_int] if not x
.empty]
100     if intervals_combined:
101         merged = pd.concat(intervals_combined, ignore_index=
True)
102     # 不去重: 每个 scope 都显示
103     save(merged, out / "quality_intervals.csv")

```

```

104     tops_combined = [x for x in [q2_top, q3_top] if not x.
empty]
105     if tops_combined:
106         save(pd.concat(tops_combined, ignore_index=True, sort=
False), out / "q4_topk.csv")
107
108     # 全族诊断 d=30（仅作参考，不进入默认结果）
109     rep.step("★ 全族 d=30 极端保守诊断（仅参考）", "3.9")
110     glob = build_global_family_diagnostic(list(records),
family_size=30)
111     glob["scope"] = glob["scope_diagnostic"]
112     save(glob, out / "
quality_intervals_global_family_d30_diagnostic.csv")
113     rep.info(f"d=30 诊断 U95 (sample) = {float(glob.iloc[0]['
U_simultaneous_95']):.5f}（与 d=3 / d=12 对照）")
114
115     # Bootstrap
116     rep.section("同设计 Parametric Bootstrap 稳定性检验", "4/4
")
117     boot = pd.DataFrame()
118     if not args.skip_bootstrap:
119         rep.step("Q2 6 情形 Bootstrap（逐级翻倍到收敛）", "4.1
")
120         for sid in Q2_SCENARIOS:
121             required = [f"Q2-S{sid}-C1", f"Q2-S{sid}-C2", f"Q2
-S{sid}-F"]
122             if all(any(r.quality_id == q for r in records) for
q in required):
123                 try:
124                     boot = pd.concat([boot,
bootstrap_stability_q2(
125                         records, sid, args.seed, args.
bootstrap_start, args.bootstrap_max,
126                         data_source=data_source,
127                         )], ignore_index=True, sort=False)

```



```

128         except ValueError as e:
129             rep.warn(f"Q2-S{sid} bootstrap skipped: {e
130 })")
131     if q3_available:
132         rep.step("Q3 Bootstrap (逐级翻倍到收敛)", "4.2")
133         try:
134             boot = pd.concat([boot, bootstrap_stability_q3
135 (
136             records, args.seed, args.bootstrap_start,
137 args.bootstrap_max,
138             data_source=data_source,
139             ignore_index=True, sort=False)
140         except ValueError as e:
141             rep.warn(f"Q3 bootstrap skipped: {e}")
142         if not boot.empty:
143             save(boot, out / "q4_bootstrap.csv")
144             print_bootstrap_table(rep, boot)
145         else:
146             rep.info("bootstrap skipped by CLI flag")
147
148     summary = {
149         "robust_q4": "completed",
150         "q2_scenarios_solved": int(q2_res["scenario_id"].
151 nunique()) if not q2_res.empty else 0,
152         "q3_solved": bool(not q3_res.empty),
153         "q2_monotonicity_pass": bool(q2_mono.empty or q2_mono[
154 "PASS"].all()),
155         "q3_monotonicity_pass": bool(q3_mono.empty or q3_mono[
156 "PASS"].all()),
157         "robust_corner_check_q2_pass": bool(rc2["PASS"].all())
158 if not rc2.empty else True,
159         "robust_corner_check_q3_pass": bool(rc3["PASS"].all())
160 if not rc3.empty else True,
161         "decode_diff_smoke_pass": (not ddiff.empty) and bool((
162 ddiff.loc[ddiff["n_changes"] > 0, "n_changes"] >= 1).all()),

```

```
154     }  
155     return summary, q2_res, q3_res, boot  
156  
157  
158 def main() -> int:
```

Listing 4: 问题四核心模型节选